

## FORMULA SHEET (FINAL)

1. The A\* algorithm is guaranteed to be optimally complete if:
  - ☐ It is always complete regardless of the heuristic used.
  - ☒ The heuristic is **optimistic**, never overestimating the cost to the goal.
  - ☐ The heuristic is **pessimistic**, never underestimating the cost to the goal.
  
2. The configuration space ( $C$ ) of a differential-drive robot constrained to move on a 2D plane has a dimensionality of:
  - ☐ 2
  - ☒ 3
  - ☐ 6
  
3. Given constant wheel velocities, the ideal motion of a differential-drive robot results in:
  - ☐ A straight line.
  - ☐ A constant curvature arc.
  - ☐ Spinning on the spot.
  - ☒ Any of the above, depending on the specific velocity ratio.
  
4. Consider a differential-drive robot spinning on the spot ( $\dot{\phi}_1 = -\dot{\phi}_2$ ). If the distance between the wheels and the robot base ( $b$ ) is **increased**, the resulting rotational velocity will be:
  - ☐ Higher.
  - ☒ Lower.
  
5. Given the wheel velocities of a differential-drive robot, we compute its body-centric velocities ( $v_x, \omega$ ) using:
  - ☒ The Forward Differential Kinematics.
  - ☐ The Inverse Differential Kinematics.
  - ☐ The Unicycle Motion Model.
  
6. A differential-drive robot has high maneuverability because it can instantly move in any direction ( $x, y$ ) within its configuration space:
  - ☐ True.
  - ☒ False.

7. Name one distinct advantage of **legged locomotion** compared to wheeled, and one advantage of **wheeled locomotion** compared to legged.

Here are the primary advantages for each type of locomotion:

### 1. Advantage of Legged Locomotion: Terrain Versatility

The most distinct advantage of legged locomotion is its ability to navigate **discontinuous and highly irregular terrain**.

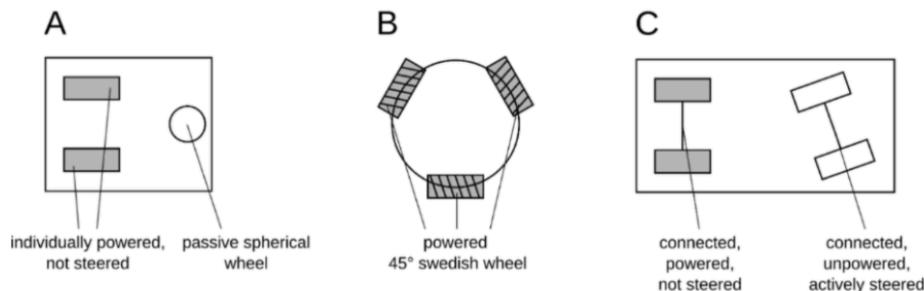
- Unlike wheels, which require a continuous path or a relatively smooth surface to maintain traction and stability, legged robots can step over obstacles, gaps, and debris.
- They use "point contacts" with the ground, allowing them to traverse environments like rocky mountainsides, deep snow, or debris-filled disaster zones that would be impassable for a wheeled vehicle.

### 2. Advantage of Wheeled Locomotion: Efficiency and Speed

The primary advantage of wheeled locomotion is its **high energy efficiency and speed** on prepared surfaces.

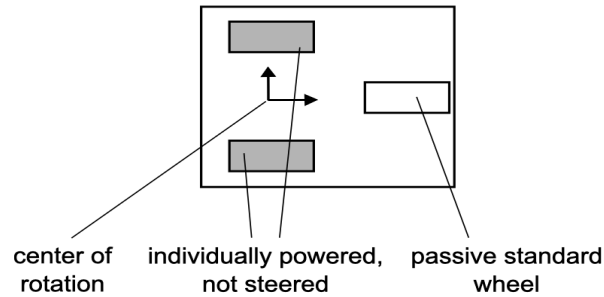
- On flat ground (like roads, warehouses, or sidewalks), a wheel is a highly efficient mechanism because it minimizes the vertical movement of the robot's center of mass.
- Legged robots must constantly expend energy to fight gravity and lift their limbs during every gait cycle. In contrast, a wheeled robot only needs to overcome rolling resistance and friction, allowing it to maintain high speeds for long durations with significantly lower battery consumption.

8. Order the mobile robot designs below by their controllability (1 = Lowest/Hardest to control, 3 = Highest/Easiest to control).



- (A) 2  
(B) 1  
(C) 3

9. Consider the robot design below with three **standard wheels** (two actively controlled, one passive). Can this robot rotate on the spot, while adhering to all wheel velocity constraints? If not, which physical constraint is violated?



No, this robot **cannot** rotate on the spot while adhering to all wheel velocity constraints.

**The Physical Constraint Violated:** The primary constraint violated is the **No-Slip Condition** (specifically, the side-slip constraint) of the **passive standard wheel**.

A standard wheel is designed to roll forward or backward; it cannot move laterally (sideways) without scrubbing or skidding. For any movement to be "ideal" (without slipping), the **Instantaneous Center of Rotation (ICR)** for the entire robot must lie along the common axis of all wheels.

- **Active Wheels:** By spinning the two powered wheels in equal and opposite directions ( $\phi_1 = -\phi_2$ ), the robot attempts to create an ICR exactly at the midpoint between them (the "center of rotation" labelled in your diagram).
- **Passive Wheel Conflict:** As shown in the diagram, the passive wheel is oriented such that its rolling axis is perpendicular to the active wheels' axis. For the robot to rotate around that center point, the passive wheel would have to move **laterally** (sideways).
- **The Result:** Because a standard wheel cannot roll sideways, it would be forced to skid or "scrub" across the floor. This violates the non-holonomic constraint that the velocity of the wheel contact point must be zero in the lateral direction.

### How to Fix the Design

To allow this robot to rotate on the spot without violating physical constraints, the passive wheel would need to be replaced with:

1. **A Castor Wheel:** Which can pivot to align itself with the direction of motion.
2. **An Omni Wheel:** Which has small rollers around the circumference to allow for passive lateral movement.

**10. In your own words, explain the difference Conditional Autonomy (Level 3) and High Autonomy (Level 4)**

The primary difference between **Conditional Autonomy (Level 3)** and **High Autonomy (Level 4)** lies in the **role of the human driver** and the system's ability to handle a "failure" or an exit from the operational design domain.

At Level 3 (**Conditional Autonomy**), the car is capable of driving itself under specific conditions (e.g., on a highway with clear lane markings). However, the human driver is still considered the **ultimate backup**.

- **The "Handover":** The driver can take their eyes off the road and perform other tasks (like checking an email), but they must remain in the driver's seat and be **ready to intervene** at any moment.
- **The Problem:** If the system encounters a situation it cannot handle (like sudden construction or heavy rain), it will issue a "Request to Intervene." The driver must be able to regain situational awareness and take control within a matter of seconds.

Level 4 (**High Autonomy**) represents a significant leap because the system is designed to handle the **entire driving task** without any human intervention within a defined area or condition (the geofence).

- **No Human Backup Required:** If the system encounters a problem or reaches the edge of its operating area, it does not rely on a human to save it. Instead, it is capable of performing a "**minimal risk maneuver**"—such as safely pulling over to the shoulder and stopping.
- **The User Experience:** In a Level 4 vehicle (like a robotaxi operating in a specific city), the "driver" is effectively a passenger. The vehicle might not even have a steering wheel or pedals, as the human is never expected to take over while the system is active.

| Feature            | Level 3 (Conditional)        | Level 4 (High)                   |
|--------------------|------------------------------|----------------------------------|
| Who is Monitoring? | The System                   | The System                       |
| Who is the Backup? | The Human Driver             | The System itself                |
| User Requirement   | Must be ready to take over.  | Can sleep or not be in the seat. |
| Failure Response   | Requests human intervention. | Performs a safe stop/pull-over.  |

**11. Explain the difference between proprioceptive and exteroceptive sensors and their respective roles in robot state estimation. Provide one example for each.**

The distinction between **proprioceptive** and **exteroceptive** sensors is fundamental to how a robot understands its state and its relationship to the world.

## 1. Proprioceptive Sensors

These sensors measure values **internal** to the robot. They monitor the robot's own movement, orientation, and mechanical state without needing to look at the environment.

- **Role in State Estimation:** They are primarily used for **dead reckoning**. By measuring things like how many times a wheel has turned or the rate of rotation, the robot can estimate its change in position ( $\Delta x$ ,  $\Delta y$ ,  $\Delta \theta$ ) over time.
- **The Downside:** Because they only measure internal changes, they are prone to **drift**. Small errors (like wheel slip) accumulate over time because the robot has no external reference to realize it has moved slightly off course.
- **Example:** An **Inertial Measurement Unit (IMU)**. It uses internal gyroscopes and accelerometers to measure the robot's angular velocity and linear acceleration.

---

## 2. Exteroceptive Sensors

These sensors measure signals from the robot's **external environment**. They allow the robot to "see" or interact with the world around it.

- **Role in State Estimation:** They are used for **localization and mapping**. By observing known landmarks, measuring the distance to walls, or receiving satellite signals, the robot can correct the drift caused by proprioceptive sensors. This provides a "ground truth" or a global reference point.
- **The Downside:** They are often computationally expensive to process and can be affected by environmental conditions (e.g., a camera failing in the dark or a GPS losing signal under a bridge).
- **Example:** A **LiDAR (Laser Rangefinder)**. It emits light pulses to measure the distance to external objects, allowing the robot to build a map of its surroundings.

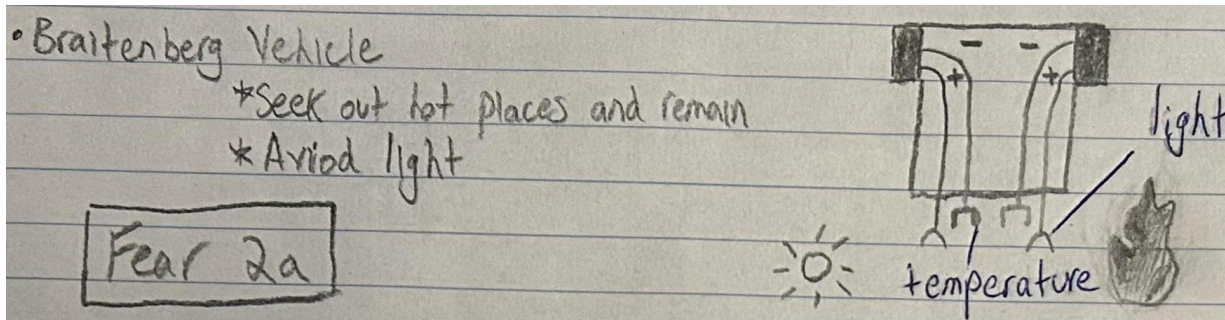
---

## Summary of Roles

In a standard state estimation filter (like a Kalman Filter), the two types work in a cycle:

1. **Proprioceptive sensors** provide the **Prediction**: "I think I moved 1 meter forward because my wheels turned twice."
2. **Exteroceptive sensors** provide the **Correction**: "Actually, I see the wall is only 0.9 meters away, so I must have slipped; I will update my position estimate."

12. Draw the wiring for a Braitenberg vehicle that seeks out hot places and aims to remain there, while simultaneously avoiding "predators" detected via light sensors.

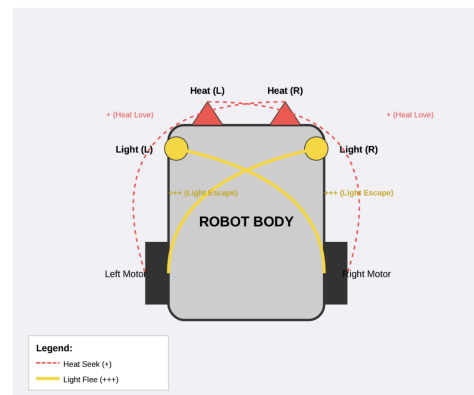


#### Wiring Specifications Recap:

- **Heat-Seeking (Type 3b "Lover"):** The heat sensors are cross-wired to the opposite motors with a standard excitatory connection (+). This ensures the robot turns toward heat and slows down only when perfectly balanced (at the source).
- **Predator-Avoidance (Type 2b "Aggressive"):** The light sensors are also cross-wired but with a significantly higher gain (+++). This causes the robot to pivot violently away from any light source, effectively escaping "predators."
- **Combined Behavior:** The summing of these signals at the motor inputs allows the robot to "stay" in warm areas while remaining highly reactive to light disturbances.

#### Braitenberg Vehicle Wiring Diagram

Heat-Seeker (Lover) + Predator-Avoider (Aggressive)



### Component List & Connection Logic

| Component          | Label | Connection Type              | Influence on Movement                                                                                      |
|--------------------|-------|------------------------------|------------------------------------------------------------------------------------------------------------|
| Right Motor        | $M_R$ | Output                       | Steers the robot <b>LEFT</b> .                                                                             |
| Left Motor         | $M_L$ | Output                       | Steers the robot <b>RIGHT</b> .                                                                            |
| Heat Sensor Left   | $H_L$ | $H_L \rightarrow M_R (+)$    | Excitatory. If $H_L$ is strong, robot turns <b>left</b> , toward the heat.                                 |
| Heat Sensor Right  | $H_R$ | $H_R \rightarrow M_L (+)$    | Excitatory. If $H_R$ is strong, robot turns <b>right</b> , toward the heat.                                |
| Light Sensor Left  | $L_L$ | $L_L \rightarrow M_R (++++)$ | <b>STRONG</b> Excitatory. If a predator is on the left, the robot violently <b>pivots right</b> to escape. |
| Light Sensor Right | $L_R$ | $L_R \rightarrow M_L (++++)$ | <b>STRONG</b> Excitatory. If a predator is on the right, the robot violently <b>pivots left</b> to escape. |

## Wiring Diagram & Component List

To design this vehicle, we must combine two separate Braitenberg behaviors: "Heat-Seeking" (excitatory) and "Light-Aversion" (excitatory or inhibitory, but we will use excitatory for a strong response). We will use a Braitenberg **Type 2b ("Aggressive")** setup, where sensors connect cross-laterally to motors.

To design this vehicle, we must combine two separate Braitenberg behaviors: "**Heat-Seeking**" (**Type 3b "Lover"**) to find and stay near warmth, and "**Light-Aversion**" (**Type 2b "Aggressive"**) to violently avoid light-based "predators." We will wire this as an excitatory (+) system, where stronger sensor signals lead to faster motor speeds.

### Wiring Diagram

Imagine this is a top-down view of the vehicle. The wiring is **cross-lateral** (left sensors connect to the right motor, and vice versa). This is critical for turning behavior. For the combined behavior, both sensor types on one side sum their signals to control the opposite motor.

Imagine the diagram is a top-down view of the vehicle, which has two wheels and four sensors mounted at the front corners.

We will define the input connection to the motor as the sum of the heat and light sensor signals on the opposite side.

## Explanation of Behaviours

- **Behaviour 1: Seeking Out and Remaining in Heat (The "Lover")**

We will wire this behaviour using an excitatory (+) connection.

1. **Seeking Heat (Type 3b):** When the robot detects a heat source to its left (H\_L is stronger), it cross-connects to the right motor (M\_R). The right wheel spins faster, causing the robot to turn left, directly toward the heat.
2. **Remaining There:** As the robot gets closer to the maximum heat, both H\_L and H\_R are strong. Both M\_R and M\_L spin at their maximum speed. Since the inputs are balanced, the robot drives forward, crashes into the heat source (or gets to the center), and "dances" around, unable to leave the high-stimulation area. It is hooked.

- **Behaviour 2: Avoiding "Predators" (The "Aggressive Escape")**

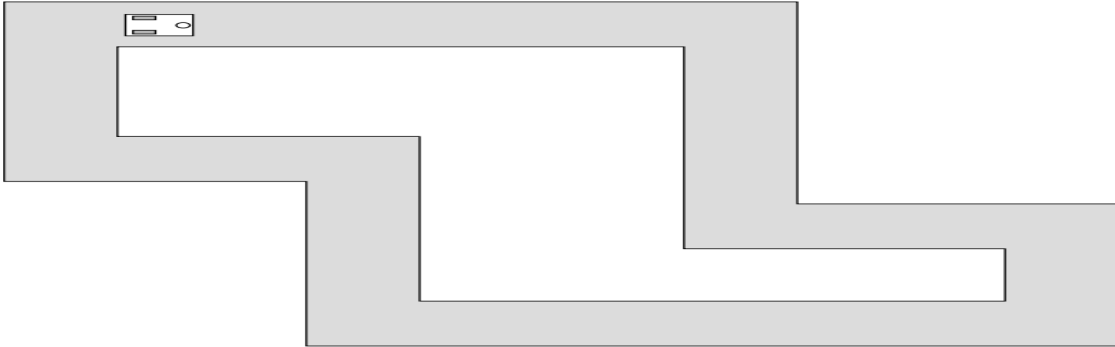
We will also wire this using an excitatory (+) connection, but a much **stronger** one.

1. **Predator Detection:** When the robot sees a bright predator (the light source) on its right (L\_R is stronger), the signal cross-connects to the left motor (M\_L).
2. **The Escape:** The left wheel spins with massive force. Because the escape connection is far stronger than the love connection, it overpowers the heat-seeking desire. This massive, unbalanced boost to the left wheel causes the robot to violently pivot away from the predator. It doesn't just flee; it turns with maximum aggression to maximize the distance.

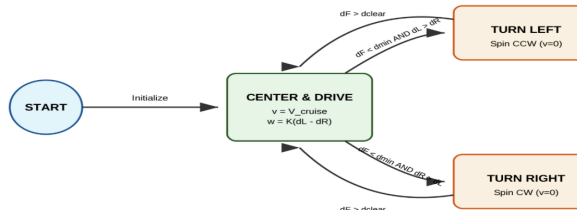
## Combined Emergent Behaviour

The vehicle will spend its time in a loving embrace with the warmest spot in the room. It will stay there perfectly happy until a predator appears. When a light is shone, the robot will experience an overpowering instinct of aggression (which manifests as a sharp turn away from the source), effectively breaking the heat-love addiction just long enough to escape to safety. Once the light is gone, it will immediately begin seeking heat again.

**13. A differential drive robot is equipped with three distance sensors, monitoring the distances in front, to the left and to the right of the robot to the closest wall. The robot is tasked to patrol a hallway, consisting of several left and right turns (see below). Draw a Finite State Machine (FSM) that implements the desired behaviour for the robot. Clearly label states and transition triggers**



**Differential Drive Robot: Hallway Patrol FSM**



**Logic:** Robot uses 3 sensors (Front, Left, Right). It centers itself during cruise using  $w = K(d_L - d_R)$ . Turns are triggered by  $d_{front} < d_{min}$  and resolved by spinning until  $d_{front} > d_{clear}$ .

FSM logic for Patrolling Behavior:

- **CENTER & DRIVE (Cruising):** This is the robot's default state while in the hallway.
  - **Action:** The robot moves forward at a cruise velocity ( $V_{cruise}$ ). It maintains its position in the center of the hallway by using its left and right sensors to adjust its angular velocity ( $w$ ) based on the difference in distances ( $d_L - d_R$ ).
- **TURN LEFT / TURN RIGHT (Cornering):** Triggered when the robot reaches a corner.
  - **Trigger:** If the front sensor detects an obstacle ( $d_F < d_{min}$ ), the robot transitions to a turn state. It chooses **Turn Left** if the left side is more open ( $d_L > d_R$ ) or **Turn Right** if the right side is more open ( $d_R > d_L$ ).
  - **Action:** The robot performs an in-place spin ( $v = 0$ ) until the front path is clear.
- **RESUMING PATROL:** Once the front sensor indicates the path is clear ( $d_F > d_{clear}$ ), the robot transitions back to the **Center & Drive** state to continue its patrol.

To enable the robot to patrol this hallway loop effectively, we can design a Finite State Machine (FSM) that prioritizes driving straight while using distance thresholds to trigger turns when obstacles are detected.

## Finite State Machine Design

The FSM consists of three primary states. We define a threshold distance,  $d_{min}$ , which represents the space needed to safely trigger a turn before hitting a wall.

| State                     | Action                                   | Transition Trigger                                        | Next State                |
|---------------------------|------------------------------------------|-----------------------------------------------------------|---------------------------|
| <b>Drive &amp; Center</b> | $v_x = V_{const}, \omega = K(d_L - d_R)$ | $Front\_Dist < d_{min}$<br>AND $Left\_Dist > Right\_Dist$ | <b>Turn Left</b>          |
| <b>Drive &amp; Center</b> | $v_x = V_{const}, \omega = K(d_L - d_R)$ | $Front\_Dist < d_{min}$<br>AND $Right\_Dist > Left\_Dist$ | <b>Turn Right</b>         |
| <b>Turn Left</b>          | $v_L = -V_{turn}, v_R = V_{turn}$        | $Front\_Dist > d_{clear}$                                 | <b>Drive &amp; Center</b> |
| <b>Turn Right</b>         | $v_L = V_{turn}, v_R = -V_{turn}$        | $Front\_Dist > d_{clear}$                                 | <b>Drive &amp; Center</b> |

To implement a robust patrolling behavior for this hallway, we can design a Finite State Machine (FSM) that focuses on centered driving and reactive turning based on sensor thresholds.

### The Logic

We define a critical distance threshold,  $d_{min}$ , which is the minimum distance allowed before the robot must stop moving forward and initiate a turn. We also use a clearance threshold,  $d_{clear}$ , to signal that the path ahead is open again.

### State Descriptions

- **Drive & Center:** This is the primary state. The robot moves forward while using its side sensors to maintain a central position in the hallway. If  $d_L > d_R$ , it steers slightly left; if  $d_R > d_L$ , it steers slightly right.
- **Turn Left / Turn Right:** When the front sensor detects a wall ( $Front\_Dist < d_{min}$ ), the robot checks which side has more "breathing room" (*Left* vs *Right*). It then performs an in-place rotation (spinning on the spot) toward the open direction.
- **Transition Back:** The robot remains in the turning state until the front sensor reports a clear path ( $Front\_Dist > d_{clear}$ ), ensuring it has fully cleared the corner before resuming forward motion.

## FSM Diagram

In the diagram below, the robot starts in the **Drive Straight** state. It constantly monitors the front distance sensor. When the front wall becomes too close, it compares the side sensors to decide which direction is "open" and transitions to the appropriate turning state.

## Explanation of States and Triggers

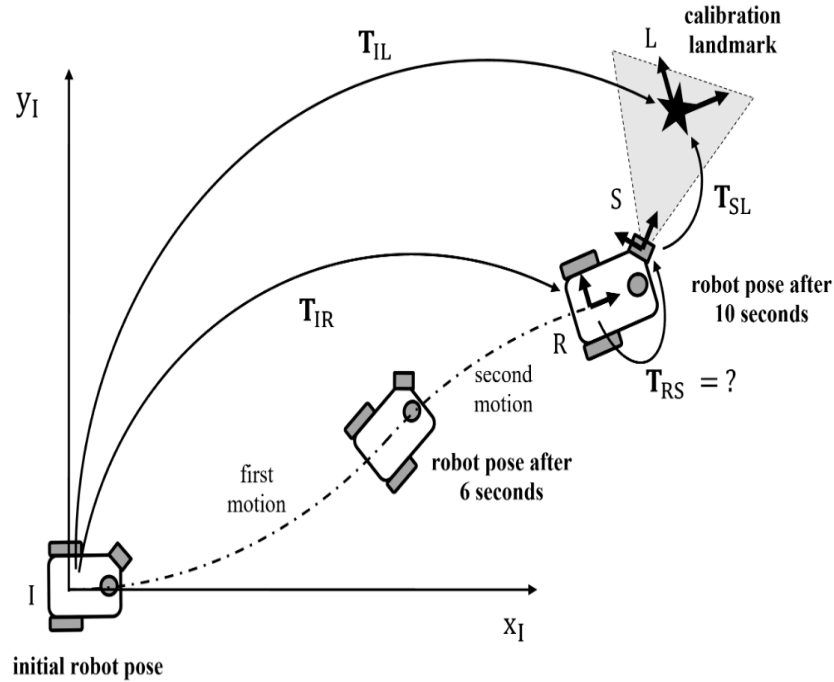
1. **Drive Straight:** This is the default state. The robot maintains equal wheel velocities to navigate the long corridors.
2. **Turn Left / Turn Right:** These are "Spin on the Spot" states. By setting the wheel velocities to be equal and opposite, the robot rotates within the hallway's width without moving forward into the wall.
3. **The Decision Logic:** The transition from "Straight" to a "Turn" is triggered by the **Front Sensor**. The specific choice between Left and Right is determined by comparing the **Left and Right Sensors**—the robot simply turns toward whichever side has more open space.
4. **Completion Trigger:** Once the robot has rotated enough that the **Front Sensor** sees a clear path again (greater than a clearance distance  $d_{clear}$ ), it transitions back to "Drive Straight."

## 2 Transformations and Mobile Robot Kinematics [25 points]

A differential-drive mobile robot has been equipped with a new camera sensor. To accurately transform camera observations into the global frame, we must determine the exact transformation matrix between the robot base and the camera sensor, denoted as  $\mathbf{T}_{RS}$ . This process is usually referred to as “calibration”.

To calibrate this unknown transform, we place a landmark (frame  $L$ ) at a known pose  $\mathbf{T}_{IL}$  in the global inertial frame (frame  $I$ ). The robot (frame  $R$ ) starts at the global origin with an initial pose  $\xi_0 = [0, 0, 0]^T$ . We drive the robot for a total of 10 seconds towards the landmark in two distinct motion segments. After the motion is complete, the camera identifies the landmark and reports its pose expressed in the sensor frame as  $\mathbf{T}_{SL}$ .

The robot is a differential drive platform with wheel radius  $r = 0.1\text{ m}$  and a track width parameter  $b = 0.2\text{ m}$  (distance from the robot center to each wheel). Each motion segment uses constant wheel velocities  $\dot{\phi}_1$  (right wheel) and  $\dot{\phi}_2$  (left wheel).



1. During the first motion sequence of 6 seconds, the robot moves with wheel velocities  $\dot{\phi}_1 = 5.0 \text{ rad/s}$  and  $\dot{\phi}_2 = 4.0 \text{ rad/s}$ . Compute the robot's resulting pose  $\xi_1 = [x_1, y_1, \theta_1]^T$  relative to the inertial frame.

#### 1. Compute Body-Centric Velocities

The linear velocity  $v$  and angular velocity  $\omega$  of the robot base are determined by the wheel radius  $r$ , the distance from center to wheels  $b$  (track width  $L = 2b$ ), and the wheel speeds  $\dot{\phi}_1, \dot{\phi}_2$ .

Given:

- $r = 0.1 \text{ m}$
- $b = 0.2 \text{ m}$  (The problem states  $b$  is center-to-wheel, so the total width  $L = 2b = 0.4 \text{ m}$ )
- $\dot{\phi}_1(\text{right}) = 5.0 \text{ rad/s}$
- $\dot{\phi}_2(\text{left}) = 4.0 \text{ rad/s}$

The equations are:

$$v = \frac{r(\dot{\phi}_1 + \dot{\phi}_2)}{2} = \frac{0.1(5.0 + 4.0)}{2} = \mathbf{0.45 \text{ m/s}}$$

$$\omega = \frac{r(\dot{\phi}_1 - \dot{\phi}_2)}{2b} = \frac{0.1(5.0 - 4.0)}{2(0.2)} = \mathbf{0.25 \text{ rad/s}}$$

#### 2. Integrate Motion Over Time

Since the velocities are constant, the robot moves in a circular arc. We integrate the velocities over the duration  $\Delta t = 6 \text{ s}$  starting from the initial pose  $\xi_0 = [0, 0, 0]^T$ .

**Final Heading ( $\theta_1$ ):**

$$\theta_1 = \theta_0 + \omega \Delta t = 0 + (0.25)(6) = \mathbf{1.5 \text{ rad}}$$

**Final Position ( $x_1, y_1$ ):**

Using the exact integration for constant  $v$  and  $\omega$ :

$$x_1 = x_0 + \int_0^{\Delta t} v \cos(\theta_0 + \omega t) dt = \frac{v}{\omega} (\sin(\theta_1) - \sin(\theta_0))$$

$$y_1 = y_0 + \int_0^{\Delta t} v \sin(\theta_0 + \omega t) dt = \frac{v}{\omega} (\cos(\theta_0) - \cos(\theta_1))$$

Substituting the values ( $\frac{v}{\omega} = \frac{0.45}{0.25} = 1.8$ ):

$$x_1 = 1.8 \cdot (\sin(1.5) - \sin(0)) \approx 1.8 \cdot (0.9975 - 0) = \mathbf{1.795 \text{ m}}$$

$$y_1 = 1.8 \cdot (\cos(0) - \cos(1.5)) \approx 1.8 \cdot (1 - 0.0707) = \mathbf{1.673 \text{ m}}$$

#### Resulting Pose $\xi_1$

After the first 6 seconds, the robot's pose relative to the inertial frame is:

$$\xi_1 = \begin{bmatrix} 1.795 \text{ m} \\ 1.673 \text{ m} \\ 1.5 \text{ rad} \end{bmatrix}$$

2. During the second motion sequence of 4 seconds, Starting immediately from pose  $\xi_1$ , the wheel velocities change to  $\dot{\phi}_1 = 4.0 \text{ rad/s}$  and  $\dot{\phi}_2 = 5.0 \text{ rad/s}$ . Compute the final global pose  $\xi_2 = [x_2, y_2, \theta_2]^T$  at  $t = 10 \text{ s}$ .

To find the final pose at  $t = 10 \text{ s}$ , we follow the same kinematic process for the second segment ( $\Delta t_2 = 4 \text{ s}$ ), starting from your previously calculated pose  $\xi_1$ .

#### 1. Compute New Body-Centric Velocities

With the wheel velocities swapped ( $\dot{\phi}_1 = 4.0, \dot{\phi}_2 = 5.0$ ), the robot is now turning in the opposite direction (left).

- Linear velocity ( $v_2$ ):

$$v_2 = \frac{r(\dot{\phi}_1 + \dot{\phi}_2)}{2} = \frac{0.1(4.0 + 5.0)}{2} = \mathbf{0.45 \text{ m/s}}$$

- Angular velocity ( $\omega_2$ ):

$$\omega_2 = \frac{r(\dot{\phi}_1 - \dot{\phi}_2)}{2b} = \frac{0.1(4.0 - 5.0)}{2(0.2)} = \mathbf{-0.25 \text{ rad/s}}$$

Substituting the values ( $\frac{v_2}{\omega_2} = \frac{0.45}{-0.25} = -1.8$ ):

- $x_2$  calculation:

$$x_2 = 1.795 + (-1.8) \cdot (\sin(0.5) - \sin(1.5))$$

$$x_2 = 1.795 - 1.8 \cdot (0.4794 - 0.9975) \approx 1.795 - 1.8 \cdot (-0.5181) = \mathbf{2.728 \text{ m}}$$

- $y_2$  calculation:

$$y_2 = 1.673 + (-1.8) \cdot (\cos(1.5) - \cos(0.5))$$

$$y_2 = 1.673 - 1.8 \cdot (0.0707 - 0.8776) \approx 1.673 - 1.8 \cdot (-0.8069) = \mathbf{3.125 \text{ m}}$$

#### 2. Integrate Motion starting from $\xi_1$

We use the pose from the first 6 seconds as our new starting point:

$$\xi_1 = [1.795, 1.673, 1.5]^T.$$

#### Final Heading ( $\theta_2$ ):

$$\theta_2 = \theta_1 + \omega_2 \Delta t_2 = 1.5 + (-0.25)(4) = \mathbf{0.5 \text{ rad}}$$

#### Final Position ( $x_2, y_2$ ):

Since  $\omega$  is constant and non-zero, we apply the integration formula for a circular arc relative to the orientation at the start of the segment:

$$x_2 = x_1 + \frac{v_2}{\omega_2} (\sin(\theta_2) - \sin(\theta_1))$$

$$y_2 = y_1 + \frac{v_2}{\omega_2} (\cos(\theta_1) - \cos(\theta_2))$$

#### Final Global Pose $\xi_2$

After the full 10 seconds of motion, the robot's coordinates in the inertial frame are:

$$\xi_2 = \begin{bmatrix} 2.728 \text{ m} \\ 3.125 \text{ m} \\ 0.5 \text{ rad} \end{bmatrix}$$

3. At the end of the motion, the system knows  $\mathbf{T}_{IR}$  (from your above calculation),  $\mathbf{T}_{IL}$  (known landmark location), and  $\mathbf{T}_{SL}$  (measured by the sensor). State the symbolic matrix equation used to compute the calibration matrix  $\mathbf{T}_{RS}$ .

To find the calibration matrix  $T_{RS}$  (the transformation from the robot base to the sensor), we use the principle of **transform chains**. We can reach the landmark frame ( $L$ ) from the inertial origin ( $I$ ) via two different paths that must be equivalent:

1. **The Known Path:**  $I \rightarrow L$  (The global pose of the landmark,  $T_{IL}$ ).
2. **The Robot-Sensor Path:**  $I \rightarrow R \rightarrow S \rightarrow L$  (The robot's pose, followed by the unknown calibration, followed by the sensor's measurement).

#### Solving for $T_{RS}$

To isolate the calibration matrix  $T_{RS}$ , we must "peel away" the other matrices using their inverses. Note that matrix multiplication is not commutative, so the order of operations is critical:

1. Multiply by  $T_{IR}^{-1}$  from the **left**:

$$T_{IR}^{-1} \cdot T_{IL} = T_{RS} \cdot T_{SL}$$

2. Multiply by  $T_{SL}^{-1}$  from the **right**:

$$\mathbf{T}_{RS} = \mathbf{T}_{IR}^{-1} \cdot \mathbf{T}_{IL} \cdot \mathbf{T}_{SL}^{-1}$$

#### The Symbolic Matrix Equation

Based on the diagram and the rules of coordinate transformations, the relationship is:

$$T_{IL} = T_{IR} \cdot T_{RS} \cdot T_{SL}$$

In this equation:

- $T_{IR}$  is the homogeneous transformation matrix derived from your calculated pose  $\xi_2$ .
- $T_{IL}$  is the known fixed pose of the landmark in the room.
- $T_{SL}$  is the measurement reported by the camera.

4. After the procedure, you discover the left wheel was spinning 10% slower than commanded. Judging from the figure above, qualitatively explain how this affects the robot's final pose relative to the landmark. Would the resulting estimate of the distance between the camera and the robot base be larger or smaller than reality?

If the left wheel was spinning 10% slower than commanded, the robot's physical trajectory would have curved more sharply to the left than the model predicted.

### Qualitative Effect on Final Pose

In both motion segments, a slower left wheel ( $\phi_2$ ) changes the velocity ratio. In the first segment, where the robot was already turning left (right wheel faster), the decreased left wheel speed would significantly **increase the curvature**, causing the robot to end up further "left" of its expected position. In the second segment, where the wheels were swapped to turn right, the slower left wheel would **impede the right turn**, causing the robot to under-rotate.

Qualitatively, the robot will be physically **closer to the landmark** and rotated further counter-clockwise than the internal state estimate ( $T_{IR}$ ) suggests.

### Effect on the Calibration Estimate ( $T_{RS}$ )

The estimate of the distance between the camera and the robot base would be **larger than reality**.

Here is the logic:

1. **The Error:** Because of the wheel slip, the robot's internal model ( $T_{IR}$ ) believes the robot is further away from the landmark than it actually is.
2. **The Measurement:** The camera ( $T_{SL}$ ) sees the "ground truth"—it correctly measures that the landmark is actually quite close.
3. **The Calibration Calculation:** The calibration math tries to bridge the gap between where the robot *thinks* it is and where the camera *sees* the landmark. To make the math work, the calculation is forced to "stretch" the transform between the robot base and the camera.

Essentially, the algorithm incorrectly attributes the robot's unexpected proximity to the landmark to a **longer physical mount** for the camera, rather than the actual cause (the wheel slip).

5. Would the described calibration procedure lead to a high-accuracy estimate of  $\mathbf{T}_{RS}$  in a real-world setting? If no, why not? Identify two sources of physical error that would influence the result.

In a real-world setting, this specific calibration procedure would likely result in a **low-accuracy** estimate of  $T_{RS}$ .

While the math is theoretically sound, the procedure relies on Open-Loop dead reckoning. This means any discrepancy between the mathematical model and the physical world accumulates over time without being corrected. Because the robot drives for a significant duration (10 seconds) before taking a single measurement, the internal estimate of its own position ( $T_{IR}$ ) will almost certainly be drifted by the time it reaches the landmark.

## Sources of Physical Error

There are two primary categories of error that would degrade the accuracy of  $T_{RS}$ :

### 1. Wheel Slip and Surface Irregularities

The kinematic model assumes a "no-slip" condition where the wheels have perfect traction. In reality, wheels encounter micro-slips due to floor texture, dust, or sudden acceleration. If the left wheel slips more than the right, the robot will end up at a different orientation than  $T_{IR}$  predicts. Since  $T_{RS}$  is calculated based on the assumption that  $T_{IR}$  is perfect, the rotational error in the robot's pose will be "baked into" the calibration matrix as a physical offset of the camera.

### 2. Mechanical Inaccuracies (Systematic Errors)

The formulas used ( $v, \omega$ ) rely on precise physical constants like the wheel radius ( $r$ ) and the track width ( $b$ ).

- **Effective Wheel Radius:** In the real world, tires compress under the weight of the robot, meaning the *effective* radius is slightly smaller than the nominal 0.1 m.
- **Uncertainty in Track Width:** The point of contact for a standard wheel is rarely a single infinitesimal point; it is a contact patch. This makes the actual distance  $b$  between the two traction points difficult to measure perfectly.
- **Impact:** These small constant errors grow linearly or quadratically with distance, ensuring that the final  $T_{IR}$  used in the calibration equation is significantly shifted from the robot's true physical location

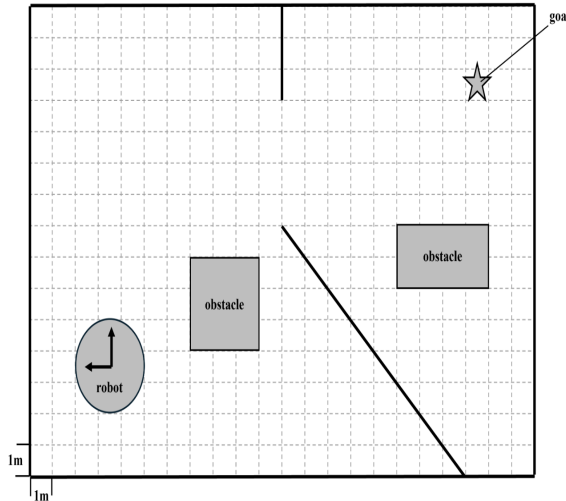
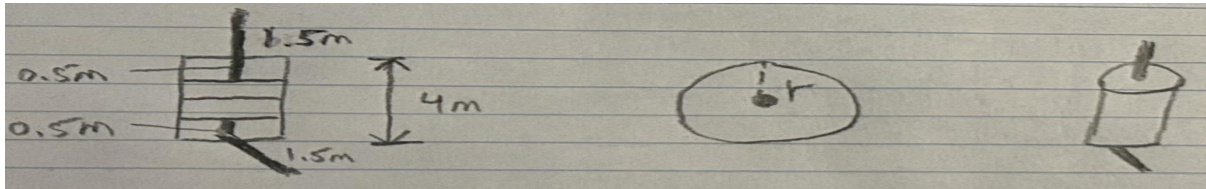
## How to improve it?

To get a high-accuracy estimate, engineers typically use **Recursive Estimation** (like an Extended Kalman Filter). Instead of driving for 10 seconds and taking one measurement, the robot would continuously observe the landmark while moving, allowing it to solve for both its own path and the camera offset simultaneously.

### 3 Path Planning and Motion Control [25 points]

Consider a differential-drive robot operating in the enclosed environment shown below. The robot can be approximated as a circle with a **radius of 1.5 meters**. The environment contains several obstacles and walls (indicated by thick black lines). The robot is tasked with planning and executing a path to the goal (indicated by the star).

1. In the provided figure, qualitatively sketch the obstructed configuration space ( $C_{obs}$ ) and the free configuration space ( $C_{free}$ ) based on the robot's dimensions.



#### How to Sketch $C_{obs}$ and $C_{free}$

To represent  $C_{obs}$  (the forbidden regions for the robot's center point), you should modify the image as follows:

1. **Inflate the Walls:** Draw a new boundary **1.5 grid units** (since 1 grid square = 1m) inside every outer wall and the internal dividing walls.
2. **Inflate the Rectangular Obstacles:** Expand the edges of the two rectangular obstacles outward by **1.5m** in every direction. The corners of these new "C-space obstacles" should be rounded arcs with a radius of 1.5m.
3. **Inflate the Diagonal Wall:** Draw parallel lines 1.5m away on both sides of the diagonal line.
4.  **$C_{obs}$  (Obstructed Space):** Shaded or colored areas representing the original obstacles plus the 1.5m "buffer" zone.
5.  **$C_{free}$  (Free Space):** The remaining white space where the robot's center can safely travel without any part of its 1.5m body hitting a wall or obstacle.

To qualitatively sketch the Configuration Space (C-space) for this robot, we use the process of **Minkowski Sums** (specifically, inflating obstacles by the robot's radius). Since the robot is a circle with a radius of **1.5m**, any point where the center of the robot is less than 1.5m from an object results in a collision.

#### Visual Characteristics of the Result

- **The Narrow Passageway:** Note the gap between the top vertical wall and the diagonal wall. In the original map, it looks wide. However, once both walls are inflated by 1.5m, the "passable" area in  $C_{free}$  will become much narrower, or potentially close entirely if the physical gap is less than 3 meters.
- **Rounded Corners:** Sharp corners on the original obstacles will become rounded in  $C_{obs}$  because the circular robot "rolls" around the corner.

#### Summary Table

| Feature                        | Physical Space ( $W$ )     | Configuration Space ( $C$ )                     |
|--------------------------------|----------------------------|-------------------------------------------------|
| <b>Robot Representation</b>    | Circle (Radius = 1.5m)     | <b>Point</b> (The center)                       |
| <b>Obstacle Representation</b> | Original thick lines/boxes | <b>Inflated versions</b> of lines/boxes         |
| <b>Passability</b>             | Looks open and spacious    | Often reveals "tight fits" or unreachable goals |

2. We now wish to use a grid-based A\* planner to solve the path planning problem with a **grid cell size of 1 meter**.

Given this grid resolution and your sketch of the obstructed and free configuration spaces, will A\* be complete? Why or why not?

Given the depicted environment, what is the maximum resolution in terms of grid cell size that would ensure completeness of the algorithm **regardless of grid alignment**?

1. Will A\* be complete?

**No, A\* will likely not be complete at a 1-meter grid resolution.**

In the context of grid-based planning, **completeness** means that if a path exists in the continuous space, the algorithm must find it. At a 1-meter resolution, the algorithm is susceptible to the **"Narrow Passage" problem**:

- **The Bottleneck:** Look at the gap between the top vertical wall and the diagonal wall. Once the robot's 1.5m radius is accounted for ( $C_{obs}$  inflation), the passable region in  $C_{free}$  becomes extremely narrow.
- **Grid Quantization:** If the grid cells are 1m wide, a cell might be marked as "obstructed" if even a small portion of it overlaps with  $C_{obs}$ . If the remaining free gap in the configuration space is smaller than the grid resolution (or poorly aligned with it), the planner will perceive the passage as "blocked," failing to find a valid path even though one technically exists in the continuous environment.

2. Maximum Resolution for Guaranteed Completeness

To ensure completeness regardless of how the grid is aligned with the environment, the grid cell size must be smaller than the narrowest part of the free configuration space.

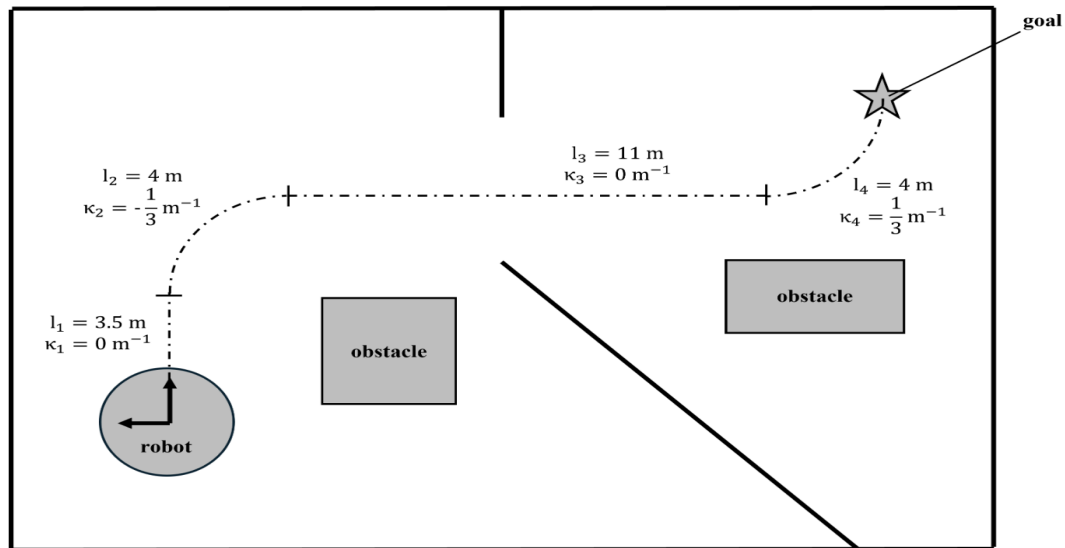
**Maximum Grid Cell Size: < 0.5 meters**

**The Reasoning:**

1. **Identify the Narrowest Gap:** The most constrained part of the map is the vertical opening near the center. The physical gap appears to be roughly **3 to 4 meters** wide.
2. **Calculate  $C_{free}$  Width:** Since the robot has a radius of 1.5m, the inflation from *both* sides of the gap subtracts 3m ( $1.5m + 1.5m$ ) from the total width. If the physical gap is 3.5m, the resulting  $C_{free}$  passage is only 0.5m wide.
3. **Resolution Requirement:** To guarantee that at least one "column" of grid cells remains entirely within  $C_{free}$  regardless of a horizontal shift (grid alignment), the cell size must be strictly smaller than the width of that passage.

**Rule of Thumb:** In a grid-based environment, to ensure a path is never "closed" by quantization, the resolution should be at least **twice as fine** as the narrowest passage you expect to navigate. Given the tight tolerances after inflation, a resolution of **0.25m to 0.5m** would be required to ensure the robot doesn't get "stuck" by its own map.

3. The planner successfully finds a path consisting of four segments: two straight lines and two curves with constant curvature (see below).



We wish to follow this path at a constant forward velocity  $v_x = 1.0 \text{ m/s}$ . Compute the sequence of required wheel velocities  $(\dot{\varphi}_1, \dot{\varphi}_2)$  for each of the four segments, considering the given lengths and curvatures of each segment in the figure above. Assume the robot has wheel radius  $r = 0.5 \text{ m}$  and track width  $b = 2 \text{ m}$ .

Diagram illustrating the relationship between curvature  $K$  and wheel velocities  $\dot{\varphi}_1, \dot{\varphi}_2$ .

For a straight line ( $K=0$ ), the required wheel velocities are  $\dot{\varphi}_1 = \dot{\varphi}_2 = \frac{v_x}{r}$ .

For a curve with curvature  $K = \frac{1}{\rho}$  (or  $K = \frac{1}{1 \text{ m}}$ ), the required wheel velocities are  $\dot{\varphi}_1 = \frac{v_x}{r}$  and  $\dot{\varphi}_2 = \frac{v_x}{r} + \omega$ , where  $\omega = K \cdot v_x$ .

①  $\omega = 9$   
 $\hookrightarrow K = \omega / v_x \Rightarrow \omega = K \cdot v_x$

②  $\dot{\varphi}_1, \dot{\varphi}_2$  - wheel velocities  
 $\hookrightarrow$  In differential Kinematics  
 $\hookrightarrow [v_x, \omega] = J(\dot{\varphi}_1, \dot{\varphi}_2)$

J - Jacobian Matrix  
 $\hookrightarrow \begin{bmatrix} \frac{r}{2} & \frac{r}{2} \\ \frac{r}{2b} & -\frac{r}{2b} \end{bmatrix}$   
 $\hookrightarrow r$  - radius  
 $\hookrightarrow b$  - length of robot center to wheel center

$\hookrightarrow v_x$  - robot body velocity  
 $\hookrightarrow \omega$  - robot angular velocity

To compute the wheel velocities for each segment, we use the inverse kinematics for a differential-drive robot. We are given the constant forward velocity  $v_x = 1.0 \text{ m/s}$ , the wheel radius  $r = 0.5 \text{ m}$ , and the track width  $b = 2.0 \text{ m}$ .

### Governing Equations

The relationship between the robot's linear velocity ( $v$ ), angular velocity ( $\omega$ ), and wheel speeds ( $\dot{\phi}_1, \dot{\phi}_2$ ) is:

1.  $v = \frac{r}{2} (\dot{\phi}_1 + \dot{\phi}_2)$
2.  $\omega = \frac{r}{2b} (\dot{\phi}_1 - \dot{\phi}_2)$

Given the curvature  $\kappa$ , the angular velocity is defined as  $\omega = v \cdot \kappa$ . We can solve for individual wheel velocities using:

$$\dot{\phi}_1 = \frac{v + \omega b}{r} = \frac{v(1 + \kappa b)}{r}$$

$$\dot{\phi}_2 = \frac{v - \omega b}{r} = \frac{v(1 - \kappa b)}{r}$$

### Segment Calculations

Using  $v = 1.0$ ,  $r = 0.5$ , and  $b = 2.0$ , the simplified formulas are:  
 $\dot{\phi}_1 = 2(1 + 2\kappa)$  and  $\dot{\phi}_2 = 2(1 - 2\kappa)$ .

#### Segment 1: Straight Line

- Parameters:  $l_1 = 3.5 \text{ m}$ ,  $\kappa_1 = 0 \text{ m}^{-1}$
- $\dot{\phi}_1 = 2(1 + 0) = \mathbf{2.0 \text{ rad/s}}$
- $\dot{\phi}_2 = 2(1 - 0) = \mathbf{2.0 \text{ rad/s}}$

#### Segment 2: Left Curve

- Parameters:  $l_2 = 4 \text{ m}$ ,  $\kappa_2 = -\frac{1}{3} \text{ m}^{-1}$
- Note: In this coordinate system, a negative curvature typically indicates a turn where the right wheel ( $\dot{\phi}_1$ ) moves faster than the left.
- $\dot{\phi}_1 = 2(1 + 2(-\frac{1}{3})) = 2(1 - \frac{2}{3}) = 2(\frac{1}{3}) = \mathbf{0.67 \text{ rad/s}}$
- $\dot{\phi}_2 = 2(1 - 2(-\frac{1}{3})) = 2(1 + \frac{2}{3}) = 2(\frac{5}{3}) = \mathbf{3.33 \text{ rad/s}}$

#### Segment 3: Straight Line

- Parameters:  $l_3 = 11 \text{ m}$ ,  $\kappa_3 = 0 \text{ m}^{-1}$
- $\dot{\phi}_1 = \mathbf{2.0 \text{ rad/s}}$
- $\dot{\phi}_2 = \mathbf{2.0 \text{ rad/s}}$

#### Segment 4: Right Curve

- Parameters:  $l_4 = 4 \text{ m}$ ,  $\kappa_4 = \frac{1}{3} \text{ m}^{-1}$
- $\dot{\phi}_1 = 2(1 + 2(\frac{1}{3})) = 2(\frac{5}{3}) = \mathbf{3.33 \text{ rad/s}}$
- $\dot{\phi}_2 = 2(1 - 2(\frac{1}{3})) = 2(\frac{1}{3}) = \mathbf{0.67 \text{ rad/s}}$

| Segment | Type     | $\kappa \text{ (m}^{-1}\text{)}$ | $\dot{\phi}_1 \text{ (Right)}$ | $\dot{\phi}_2 \text{ (Left)}$ | Duration ( $l/v$ ) |
|---------|----------|----------------------------------|--------------------------------|-------------------------------|--------------------|
| 1       | Straight | 0                                | 2.0 rad/s                      | 2.0 rad/s                     | 3.5 s              |
| 2       | Curve    | $-1/3$                           | 0.67 rad/s                     | 3.33 rad/s                    | 4.0 s              |
| 3       | Straight | 0                                | 2.0 rad/s                      | 2.0 rad/s                     | 11.0 s             |
| 4       | Curve    | $1/3$                            | 3.33 rad/s                     | 0.67 rad/s                    | 4.0 s              |

4. To ensure the robot maintains  $1.0\text{ m/s}$ , you implement a closed-loop **P-controller** to regulate wheel speeds. During testing, you observe that the robot never fully reaches  $1.0\text{ m/s}$ , but stabilizes at a slightly lower speed (e.g.,  $0.95\text{ m/s}$ ).

In your own words, explain the reason for this occurrence.

Conceptually, how should the control law be modified to eliminate this offset?

This phenomenon is known as **Steady-State Error**, and it is a classic characteristic of a purely proportional control system.

### Why the Offset Occurs

In a **Proportional (P) controller**, the control signal (the power sent to the motors) is directly proportional to the error (the difference between the target speed and the actual speed).

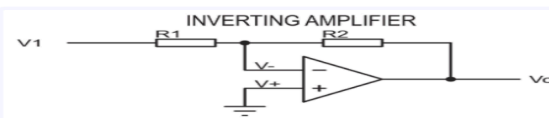
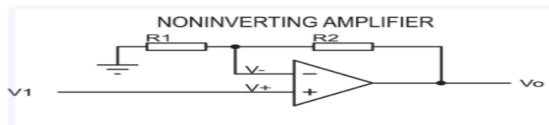
As the robot accelerates and gets closer to  $1.0\text{ m/s}$ , the error becomes very small. Consequently, the controller reduces the power output. At  $0.95\text{ m/s}$ , the robot reaches an equilibrium: the small amount of power generated by the remaining  $0.05\text{ m/s}$  error is exactly enough to overcome internal physical resistances, such as **friction, gear damping, and rolling resistance**, but not enough to accelerate the robot any further. To the P-controller, the system looks stable, even though it hasn't reached the target.

### How to Eliminate the Offset

To eliminate this offset, the control law should be modified by adding an **Integral (I) term**, transforming it into a **PI Controller**.

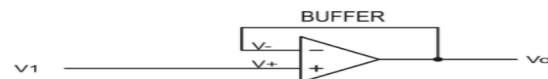
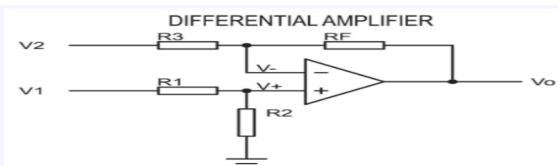
**Conceptual Modification:** The Integral term "remembers" the error over time. While the P-term becomes satisfied with a small error, the I-term sees that the error is persisting and begins to wind up or accumulate. This accumulation continuously increases the command signal sent to the motors even if the error is small. By integrating the error, the controller will keep pushing more power to the wheels until the error is exactly zero. Only when the actual speed is  $1.0\text{ m/s}$  does the Integral term stop increasing, effectively finding the exact amount of constant power needed to maintain that speed against friction.

A noninverting amplifier is an op-amp circuit that outputs an amplified in-phase signal. This circuit also acts as a voltage follower circuit. The gain of this circuit is determined by the values of  $R_1$  and  $R_2$ .



An inverting amplifier is an op-amp circuit that outputs an amplified out-of-phase signal. The gain of this circuit is also controlled by the values of  $R_1$  and  $R_2$ . One of the main applications of this op-amp circuit is in audio mixers.

A differential amplifier is a type of electronic amplifier that multiplies the difference between two input voltages but suppresses any voltage common to the two inputs. A very common application for this type of amplifier is to remove noise from signals. They can also be used to interface with sensors or as a part of a feedback system.



The image shows the most common Op-Amp circuit configurations. The configurations shown are the Noninverting Amplifier, the Differential Amplifier, the Inverting Amplifier, and the Buffer. Op-Amps are differential voltage amplifiers, meaning they are designed to amplify the difference in voltage between the two input terminals.

## 4 Robot Localization and Mapping [25 points]

Consider a warehouse robot moving in 1D along a conveyor belt. To estimate its one-dimensional position in  $x$ , it uses its motion model and bearing measurements to a known landmark  $L$  at  $(l_x, l_y)$ . The robot's motion model depends on its velocity  $u$ , which is the control input to the motion model. The robot can directly measure this velocity.

The discrete time motion model is given as

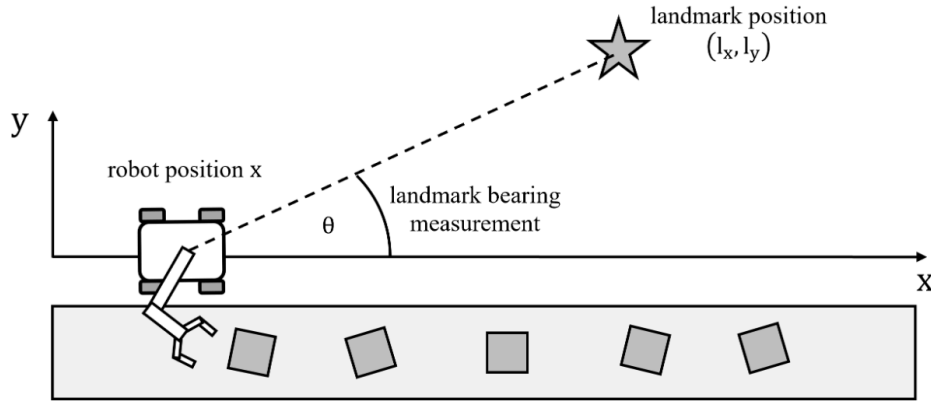
$$x(t_{k+1}) = x(t_k) + \Delta t \cdot u(t_k).$$

The bearing observation model is given as

$$z = \theta = h(x) = \tan^{-1} \left( \frac{l_y}{l_x - x} \right).$$

The observation Jacobian can be computed as

$$H(x) = \frac{\partial h(x)}{\partial x} = \frac{l_y}{(l_x - x)^2 + l_y^2}.$$



1. **Prediction:** The robot starts at  $x_{\text{post}}(t_0) = 1.0 \text{ m}$  with variance  $C_{\text{post}}(t_0) = 1.0 \text{ m}^2$ . It then moves for  $\Delta t = 2 \text{ s}$  with a measured constant speed of  $u = 1.0 \text{ m/s}$ . The uncertainty of this measured control input is  $C_u = 1.0 \text{ m}^2/\text{s}^2$ . Calculate the predicted state  $x_p(t_1)$  and variance  $C_p(t_1)$ .

To calculate the predicted state and its variance, we apply the **Prediction Step** of an Extended Kalman Filter (EKF) using the provided discrete-time motion model.

### 1. Predicted State $x_p(t_1)$

The predicted state is calculated by propagating the previous estimate through the motion model.

Given:

- Initial position:  $x_{\text{post}}(t_0) = 1.0 \text{ m}$
- Time step:  $\Delta t = 2 \text{ s}$
- Measured velocity:  $u(t_0) = 1.0 \text{ m/s}$

Calculation:

$$x_p(t_1) = x_{\text{post}}(t_0) + \Delta t \cdot u(t_0)$$

$$x_p(t_1) = 1.0 + (2 \cdot 1.0) = \mathbf{3.0 \text{ m}}$$

Identifying the terms:

- Initial Variance:**  $C_{\text{post}}(t_0) = 1.0 \text{ m}^2$
- State Transition ( $A$ ):** Since  $x_{k+1}$  depends directly on  $x_k$ , the Jacobian  $A = \frac{\partial x_{k+1}}{\partial x_k} = 1$ .
- Process Noise ( $Q$ ):** This represents the uncertainty added by the velocity measurement over the time step. According to the error propagation of  $x = \Delta t \cdot u$ :

$$Q = \text{Var}(\Delta t \cdot u) = \Delta t^2 \cdot C_u$$

Given  $C_u = 1.0 \text{ m}^2/\text{s}^2$  and  $\Delta t = 2 \text{ s}$ :

$$Q = 2^2 \cdot 1.0 = 4.0 \text{ m}^2$$

Calculation:

$$C_p(t_1) = (1)^2 \cdot C_{\text{post}}(t_0) + Q$$

$$C_p(t_1) = 1.0 + 4.0 = \mathbf{5.0 \text{ m}^2}$$

## Summary of Results

At time  $t_1$ , the robot's predicted state and uncertainty are:

### 2. Predicted Variance $C_p(t_1)$

The predicted variance accounts for the uncertainty of the initial position and the noise introduced by the control input (velocity) during the movement.

For a linear motion model  $x_{k+1} = Ax_k + Bu_k$ , the variance propagates as:

$$C_p(t_1) = AC_{\text{post}}(t_0)A^T + Q$$

$$\bullet \quad \mathbf{x_p(t_1) = 3.0 \text{ m}}$$

$$\bullet \quad \mathbf{C_p(t_1) = 5.0 \text{ m}^2}$$

### ■ Motion Model:

$$\hookrightarrow x_{k+1} = g(x_k, u_k)$$

$$= x_k + \Delta t \cdot u_k$$

\*  $u_k$  - velocity

### ■ Initial State:

$$\hookrightarrow \bar{x}_{\text{post}}(t_0) = 1\text{m (old state mean)}$$

$$\hookrightarrow C_{\text{post}}(t_0) = 1\text{m}^2 \text{ (old state covariance)}$$

• NOTE: One-dimensional (1-D)

► All of Section 4

### ■ Control Inputs:

$$\hookrightarrow u_k = \frac{1\text{ m}}{\text{s}}$$

\* velocity (mean)

$$\hookrightarrow C_u = \frac{1\text{ m}^2}{\text{s}^2}$$

\* Covariance

$$\hookrightarrow \Delta t = 2\text{s}$$

### ■ Next Time ( $t_1$ ): Prediction

$$\hookrightarrow \bar{x}_p(t_1) = g(\bar{x}_{\text{post}}(t_0), u_k)$$

### ■ Correction:

$$\hookrightarrow \bar{x}_{\text{post}}(t_1) = \bar{x}_p(t_1) + K(\bar{z} - h(\bar{x}_p(t_1)))$$

$$\hookrightarrow C_{\text{post}}(t_1) = (1 - KH) C_p(t_1)$$

• Check:

$$\triangleright K = (C_p^{-1} + H^T R^{-1} H)^{-1} H^T R^{-1}$$

$$= \bar{x}_{\text{post}}(t_0) + \Delta t \cdot u_k$$

$$= 1\text{m} + (2\text{s})(\frac{1\text{m}}{\text{s}})$$

$$= \boxed{3\text{m}}$$

\*  $\bar{x}_p(t_1)$  - mean

$$\hookrightarrow C_p(t_1) = G_{\bar{x}} C_{\text{post}}(t_0) G_{\bar{x}}^T + G_u C_u G_u^T$$

$$= (1)(1)(1) + (2\text{s})(1)(2\text{s})$$

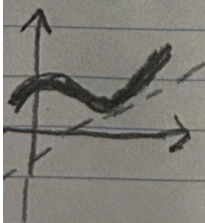
$$= \boxed{5\text{m}^2}$$

\*  $C_p(t_1)$  - uncertainty

### ■ Jacobians

$$\hookrightarrow G_{\bar{x}} = \frac{\partial g}{\partial x_k} = 1$$

$$\hookrightarrow G_u = \frac{\partial g}{\partial u_k} = \Delta t = 2\text{s}$$



NOTE:  $G_{\bar{x}}^T$  &  $G_u^T$  are the transpose (T)

► rows  $\rightarrow$  columns  $\leftrightarrow$  columns  $\rightarrow$  rows

### ■ Motion Model:

$$\hookrightarrow g(x(t_0), u(t_0)) = x(t_1) = x(t_0) + \Delta t \cdot u(t_0)$$

**2. Correction:** At the predicted position, the robot measures the bearing  $\theta$  to the landmark at  $(l_x = 5, l_y = 2)$ . The measurement is  $z = \theta = 30^\circ$  ( $0.52 \text{ rad}$ ) with noise variance  $R = 0.5 \text{ rad}^2$ . Compute the corrected posterior estimate  $x_{\text{post}}(t_1)$  and variance  $C_{\text{post}}(t_1)$ .

To compute the corrected posterior estimate and variance, we perform the **Correction (Update) Step** of the Extended Kalman Filter (EKF). We use the predicted state  $x_p(t_1) = 3.0$  m and predicted variance  $C_p(t_1) = 5.0 \text{ m}^2$  calculated previously.

#### 1. Compute the Innovation and Jacobian

First, we determine what the robot *expected* to see versus what it actually measured.

##### Given Parameters:

- Landmark position:  $(l_x, l_y) = (5, 2)$
- Measurement:  $z = 0.52 \text{ rad}$
- Measurement Noise:  $R = 0.5 \text{ rad}^2$

##### Expected Measurement $h(x_p)$ :

$$h(3.0) = \tan^{-1}\left(\frac{2}{5 - 3.0}\right) = \tan^{-1}(1) = \mathbf{0.785 \text{ rad}}$$

##### Innovation ( $y$ ):

$$y = z - h(x_p) = 0.52 - 0.785 = \mathbf{-0.265 \text{ rad}}$$

##### Observation Jacobian ( $H$ ):

Using the provided formula at  $x_p = 3.0$ :

$$H = \frac{2}{(5 - 3.0)^2 + 2^2} = \frac{2}{4 + 4} = \mathbf{0.25 \text{ m}^{-1}}$$

#### 2. Compute the Kalman Gain ( $K$ )

The Kalman Gain determines how much we trust the new camera measurement relative to our wheel-odometry prediction.

$$K = C_p H^T (H C_p H^T + R)^{-1}$$

$$K = 5.0(0.25) \cdot [0.25(5.0)(0.25) + 0.5]^{-1}$$

$$K = 1.25 \cdot [0.3125 + 0.5]^{-1} = \frac{1.25}{0.8125} \approx \mathbf{1.538}$$

#### 3. Calculate Posterior Estimate $x_{\text{post}}(t_1)$ and Variance $C_{\text{post}}(t_1)$

##### Posterior State:

The robot corrects its position based on the "bearing error":

$$x_{\text{post}} = x_p + K(y)$$

$$x_{\text{post}} = 3.0 + 1.538(-0.265) \approx 3.0 - 0.407 = \mathbf{2.593 \text{ m}}$$

##### Posterior Variance:

The observation reduces the robot's uncertainty:

$$C_{\text{post}} = (I - KH)C_p$$

$$C_{\text{post}} = (1 - 1.538 \cdot 0.25) \cdot 5.0$$

$$C_{\text{post}} = (1 - 0.3845) \cdot 5.0 = 0.6155 \cdot 5.0 = \mathbf{3.078 \text{ m}^2}$$

## Summary of Results

After incorporating the bearing measurement, the robot's state is:

- $\mathbf{x_{\text{post}}(t_1) = 2.593 \text{ m}}$
- $\mathbf{C_{\text{post}}(t_1) = 3.078 \text{ m}^2}$

### 3. Based on $H(x)$ , are bearing measurements more informative when the robot is **closer** to the landmark or **further away**? Justify using the relationship between the observation Jacobian $H(x)$ and update equation for the posterior uncertainty.

Bearing measurements are significantly **more informative** when the robot is **closer** to the landmark.

To justify this, we look at the mathematical relationship between the observation Jacobian  $H(x)$ , the landmark distance, and the reduction of posterior uncertainty.

#### 1. The Relationship in $H(x)$

The observation Jacobian  $H(x)$  represents the sensitivity of the measurement (the bearing  $\theta$ ) to changes in the robot's state ( $x$ ). Given:

$$H(x) = \frac{l_y}{(l_x - x)^2 + l_y^2}$$

As the robot moves closer to the landmark along the x-axis, the term  $(l_x - x)$  decreases. Since this term is in the denominator, a smaller distance results in a **larger value for  $H(x)$** .

Physically, this means that when you are close to an object, moving one inch causes a large shift in the angle (bearing) you see it at. When you are far away, moving that same inch barely changes the angle at all.

### Summary

When the robot is far away,  $H(x)$  is small,  $K$  is small, and the bearing measurement does very little to correct the robot's position. When the robot is close,  $H(x)$  is large, the measurement provides a high-resolution "anchor," and the **posterior uncertainty is significantly reduced**, making the measurement much more informative.

#### 2. Impact on Posterior Uncertainty

In an EKF, the posterior uncertainty  $C_{post}$  is calculated as:

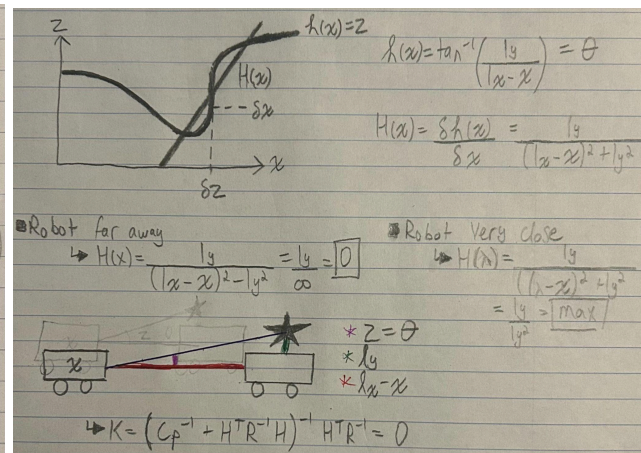
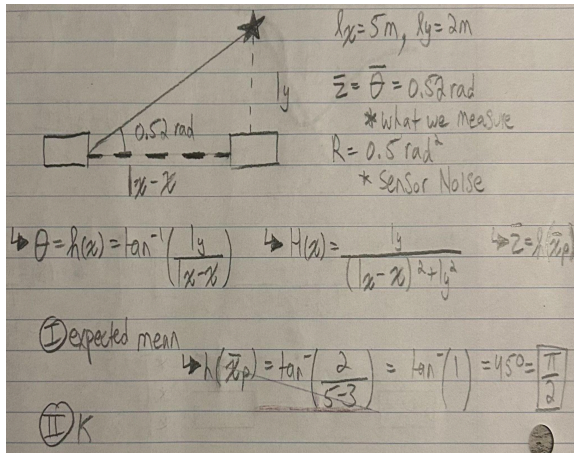
$$C_{post} = (I - KH)C_p$$

The amount of uncertainty "removed" from the system depends on the product  $KH$ . We can further examine the Kalman Gain  $K$ :

$$K = \frac{C_p H}{H^2 C_p + R}$$

When  $H$  is large (robot is close):

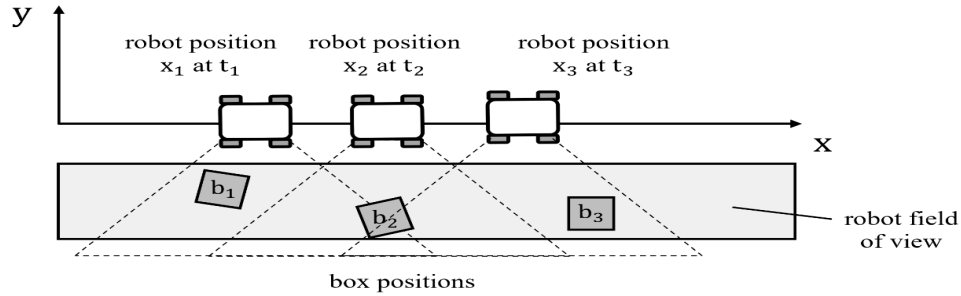
- The Kalman Gain  $K$  increases because the measurement is seen as highly sensitive and reliable for determining position.
- The term  $(I - KH)$  becomes smaller.
- Consequently,  $C_{post}$  **becomes much smaller** than the predicted uncertainty  $C_p$ .



4. In an update to the warehouse infrastructure, the landmark has been removed. Instead, the robot is now equipped with a camera observing fixed boxes  $b_1, b_2, b_3$  on the conveyor belt. Using this camera, the robot is tasked to localize its own position  $x$  over time, while also determining the unknown positions of the fixed boxes. This leads to a simultaneous localization and mapping (SLAM) problem, where the boxes are landmarks to be mapped. The robot moves through three positions  $(x_1, x_2, x_3)$  as shown below. It is still able to measure its velocity during motion.

During motion, the following observations are made:

- At  $x_1$ , the robot observes boxes  $\{b_1, b_2\}$ .
- At  $x_2$ , the robot observes boxes  $\{b_2\}$ .
- At  $x_3$ , the robot observes boxes  $\{b_2, b_3\}$ .



Draw the **Factor Graph** and sketch the **Information Matrix (A)** sparsity pattern for the system vector  $[x_1, x_2, x_3, b_1, b_2, b_3]^T$  resulting for this SLAM problem.

#### Factor Connections:

1. **Odometry:**  $f_{u1}(x_1, x_2)$  and  $f_{u2}(x_2, x_3)$ .
2. **Observations at  $x_1$ :**  $f_{z1}(x_1, b_1)$  and  $f_{z2}(x_1, b_2)$ .
3. **Observations at  $x_2$ :**  $f_{z3}(x_2, b_2)$ .
4. **Observations at  $x_3$ :**  $f_{z4}(x_3, b_2)$  and  $f_{z5}(x_3, b_3)$ .

#### 2. Information Matrix (A) Sparsity Pattern

The Information Matrix  $\Lambda$  (often associated with the Hessian in least-squares SLAM) has a non-zero entry at row  $i$ , column  $j$  if there is a factor connecting variable  $i$  and variable  $j$ . The diagonal is always non-zero.

For the state vector  $\Theta = [x_1, x_2, x_3, b_1, b_2, b_3]^T$ :

To solve this SLAM problem, we represent the relationships between the robot poses, the landmarks (boxes), and the sensor measurements using a factor graph. From that graph, we can derive the structure of the Information Matrix.

#### 1. The Factor Graph

In a factor graph, **variable nodes** (circles) represent the unknown states we want to estimate, and **factor nodes** (small squares) represent the probabilistic constraints (measurements) between them.

- **Variables:**  $x_1, x_2, x_3$  (robot poses) and  $b_1, b_2, b_3$  (box positions).
- **Odometry Factors ( $f_u$ ):** Connect consecutive poses ( $x_1 \rightarrow x_2$  and  $x_2 \rightarrow x_3$ ) based on measured velocity.
- **Observation Factors ( $f_z$ ):** Connect a pose to a box if the robot saw that box at that specific time.

|       | $x_1$ | $x_2$ | $x_3$ | $b_1$ | $b_2$ | $b_3$ | Reason for connection                                                                 |
|-------|-------|-------|-------|-------|-------|-------|---------------------------------------------------------------------------------------|
| $x_1$ | ■     | ■     |       | ■     | ■     |       | Odometry ( $x_1, x_2$ ), Observations ( $x_1, b_1$ ), Observations ( $x_1, b_2$ )     |
| $x_2$ | ■     | ■     | ■     |       | ■     |       | Odometry ( $x_1, x_2$ ), Odometry ( $x_2, x_3$ ), Observations ( $x_2, b_2$ )         |
| $x_3$ |       | ■     | ■     |       | ■     | ■     | Odometry ( $x_2, x_3$ ), Observations ( $x_3, b_2$ ), Observations ( $x_3, b_3$ )     |
| $b_1$ | ■     |       |       | ■     |       |       | Observations ( $x_1, b_1$ )                                                           |
| $b_2$ | ■     | ■     | ■     |       | ■     |       | Observations ( $x_1, b_2$ ), Observations ( $x_2, b_2$ ), Observations ( $x_3, b_2$ ) |
| $b_3$ |       |       | ■     |       |       | ■     | Observations ( $x_3, b_3$ )                                                           |

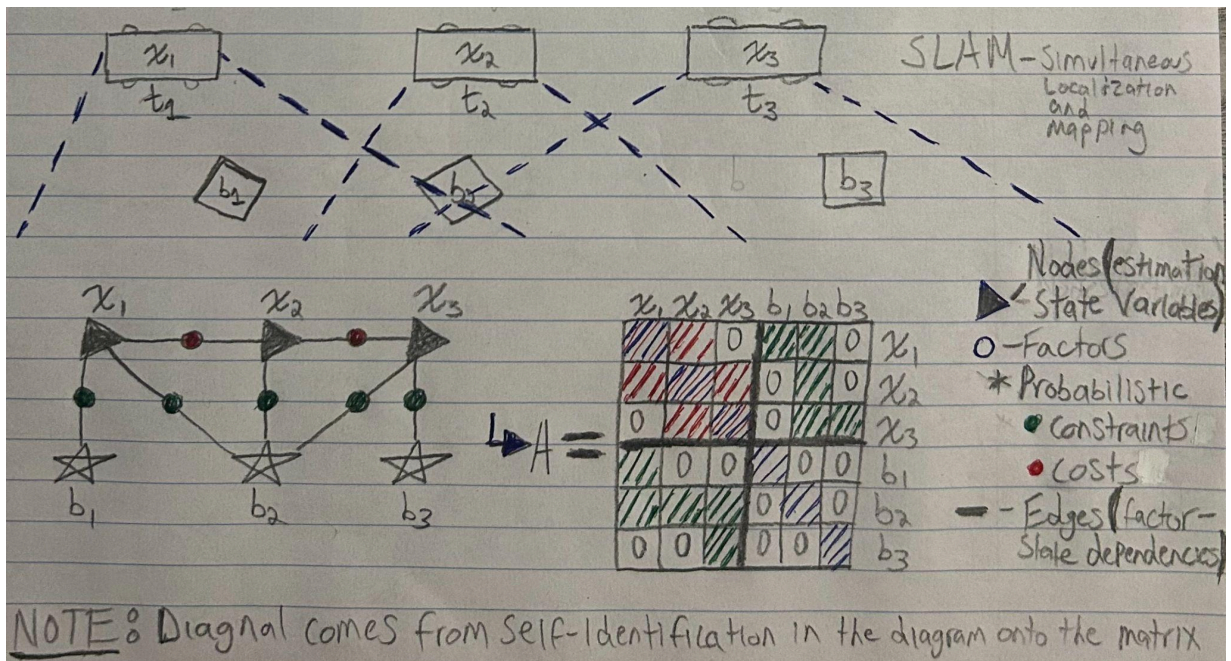
## Sparsity Sketch:

### Plaintext

|    | x1  | x2 | x3 | b1 | b2 | b3 |
|----|-----|----|----|----|----|----|
| x1 | [ X | X  | .  | X  | X  | .  |
| x2 | [ X | X  | X  | .  | X  | .  |
| x3 | [ . | X  | X  | .  | X  | X  |
| b1 | [ X | .  | .  | X  | .  | .  |
| b2 | [ X | X  | X  | .  | X  | .  |
| b3 | [ . | .  | X  | .  | .  | X  |

## Observations on Sparsity

- **The Pose Chain:** The upper-left  $3 \times 3$  block forms a tridiagonal band because of the sequential odometry ( $x_1 - x_2 - x_3$ ).
- **The Landmark Block:** The bottom-right  $3 \times 3$  block is diagonal. Landmarks are not directly connected to each other; they only "talk" to each other through the robot's poses.
- **The "Star" Landmark:** Notice that  $b_2$  has the most connections (3 robot poses). In the matrix, this creates a dense column/row that couples all robot poses together, which is the core of how SLAM improves the entire trajectory estimate simultaneously.



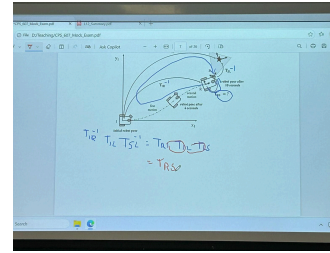
- One-dimensional Problems (focused)

### curvature-velocity mapping

$$\kappa = \frac{\omega}{v_x} \rightarrow \omega = \kappa v_x$$

### inverse differential kinematics

$$\begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \end{bmatrix} = \begin{bmatrix} 1/r & b/r \\ 1/r & -b/r \end{bmatrix} \begin{bmatrix} v_x \\ \omega \end{bmatrix}$$



L12\_Summary.pdf

## Extended Kalman Filter: Prediction and Correction

- The Extended Kalman Filter allows us to iteratively estimate the state of our robot using the following procedure

### 1. Prediction Step (derived last week)

$$\mathbf{x}_p(t_{k+1}) \sim \mathcal{N}(\mathbf{g}(\bar{\mathbf{x}}_{\text{post}}(t_k), \mathbf{u}(t_k)), \mathbf{G}_x \mathbf{C}_{\text{post}}(t_k) \mathbf{G}_x^T + \mathbf{G}_u \mathbf{C}_u \mathbf{G}_u^T)$$

Mean of new state using motion model  $\mathbf{g}(\mathbf{x}, \mathbf{u})$       Last state uncertainty passed through motion model      Control input uncertainty passed through motion model

### 2. Correction Step (derived this week)

$$\mathbf{K} = (\mathbf{C}_p^{-1}(t_{k+1}) + \mathbf{H}^T \mathbf{R}^{-1} \mathbf{H})^{-1} \mathbf{H}^T \mathbf{R}^{-1}$$

Compute Kalman gain to determine weighting between prior and measurement

$$\bar{\mathbf{x}}_{\text{post}}(t_{k+1}) = \bar{\mathbf{x}}_p(t_{k+1}) + \mathbf{K}(\bar{\mathbf{z}} - \mathbf{h}(\bar{\mathbf{x}}_p(t_{k+1})))$$

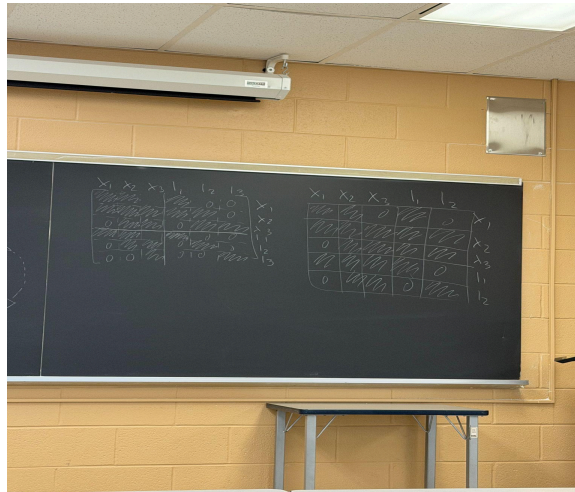
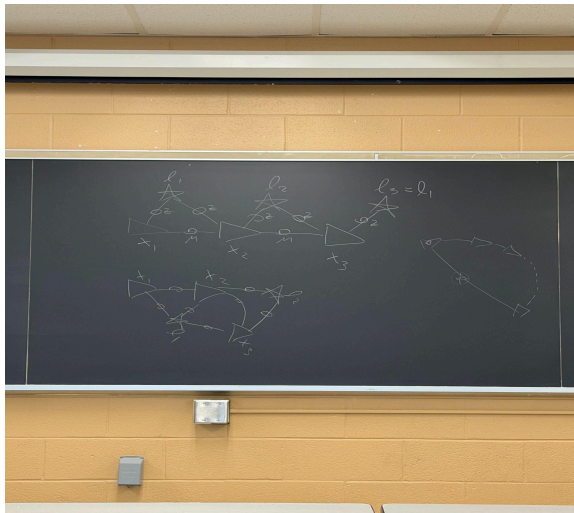
Update mean of state estimate

$$\mathbf{C}_{\text{post}}(t_{k+1}) = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{C}_p(t_{k+1})$$

Update covariance of state estimate

Note: Here  $\mathbf{I}$  is the identity matrix.

- Quiz (Factor Graph)





## Quiz 10: Review

### Question 1 (1 point) ✓ Saved

A differential drive robot is commanded to drive in a straight line at 0.5m/s. If the left wheel encounters a patch of ice and experiences 20% slip (rotates faster than the physical displacement it creates), the robot will:

- ☐ Veer to the left.
- ☒ Veer to the right.
- ☐ Continue straight but at a slower speed.
- ☐ Continue straight but at a faster speed.

### Question 2 (1 point) ✓ Saved

The dimension of the Configuration Space for a standard differential drive robot is lower than the dimension of the Configuration Space for an omnidirectional robot (e.g., one using Swedish wheels).

- ☐ True
- ☒ False

### Question 3 (1 point) ✓ Saved

Match each sensor to its corresponding classification:

|     |                           |                          |
|-----|---------------------------|--------------------------|
| 1 ▾ | Wheel Encoder             |                          |
| 1 ▾ | IMU                       | 1. Proprioceptive Sensor |
| 2 ▾ | Laser Rangefinder (Lidar) | 2. Exteroceptive Sensor  |
| 2 ▾ | GPS                       |                          |

**Question 4** (1 point) ✓ *Saved*

To treat a circular robot as a single point for path planning, we must "inflate" the obstacles in the map by the diameter of the robot.

☐ True

☒ False

**Question 5** (1 point) ✓ *Saved*

You are using A\* to navigate a grid-based map where the robot can only move Up, Down, Left, and Right (no diagonals). Select all suitable distance heuristics such that A\* is guaranteed to find the optimal path.

☒ Zero Distance

$$h = 0$$

☒ Euclidean Distance

$$h = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

☒ Manhattan Distance

$$h = |x_1 - x_2| + |y_1 - y_2|$$

**Question 6** (1 point) ✓ *Saved*

You are using a standard Rapidly-exploring Random Tree (RRT) to find a path through a large warehouse. Which of the following is a true characteristic of the path generated by RRT compared to  $A^*$ ?

- ☐ RRT will likely find a shorter path than  $A^*$ .
- ☐ RRT will find the same path as  $A^*$ , if we run it for an infinite time.
- ☒ RRT will likely produce a longer and less optimal path than  $A^*$ .

**QUESTION 7**

Steady-state error is the term for the 0.05m difference you are seeing. The control output of a pure P-controller is proportional to the error; when the error approaches zero, the control output also approaches zero. To maintain a position, however, many real-world systems need a non-zero control effort (e.g., a continuous steering angle to offset a physical bias). The robot settles at an offset where the push from the error precisely balances the external pressures or system requirements, since a P-controller only exerts that effort when an error occurs.

Although this offset can be reduced by increasing the proportional gain ( $K_p$ ), setting it to infinity is a bad idea because it causes oscillation and instability. The robot overshoots the target distance as  $K_p$  it rises because its corrections grow more forceful. As a result, the robot zigzags or snakes along the hallway in an underdamped system. These oscillations intensify at very high gains until the system becomes unstable, which could result in the robot jerking violently or colliding with the wall.

Furthermore, sensor noise and actuator saturation are physical phenomena that infinite gain ignores. Real motors have a maximum speed; once this is reached, increasing the gain does not yield any further benefits, hence reducing the controller to a rudimentary on-off switch. Furthermore, every sensor has a tiny bit of jitter. This small noise will be multiplied by a large  $K_p$  number into large, high-frequency steering commands, which will cause considerable mechanical wear and battery drain. The conventional approach is to add an Integral ( $K_i$ ) term, which accumulates the mistake over time and provides the required push to safely erase the steady-state offset, rather than expanding  $K_p$  to infinity.

**QUESTION 8**

The area of a robot's uncertainty ellipse will steadily grow over time when it just uses wheel encoders and its internal motion model, a procedure known as dead reckoning. Every movement in a probabilistic framework, such as the Extended Kalman Filter (EKF) includes a prediction step that contributes process noise to the state estimation. Stochastic flaws like small wheel slippage, encoder quantization, and uneven floor surfaces are all taken into account by this noise. Since there is no way to reduce or constrain this uncertainty in the absence of

additional sensor measurements (such as LiDAR or GPS) to give a correction step, the covariance will increase endlessly.

This ellipse's growth is caused by both the compounding effect of orientation mistakes and the distance travelled. A small initial orientation mistake functions as an angular offset that spreads out with time because the robot's heading ( $\theta$ ) is also predicted using the same noisy encoders. A 1-degree heading error causes a lateral position error that increases linearly with the distance travelled, even if the robot thinks it is moving in a perfectly straight line. As a result, the uncertainty ellipse usually extends into an elongated shape, representing a huge cone of potential locations for the robot.

This shows a lack of information acquisition in terms of mathematics. The system's confidence always declines because the Information Matrix is the inverse of the Covariance Matrix, and no fresh information is being obtained from the environment to stiffen the limitations of the robot's location. This is physically comparable to someone attempting to walk in a straight line while wearing a blindfold; with each step, their mental map of where they are becomes more hazy and unreliable until they are able to touch a wall or open their eyes to recalibrate.

**Question 9** (1 point) ✓ *Saved*

The Kalman Gain of an EKF will be larger, if the sensor noise of our observation during the correction step is higher.

- ☐ True
- ☒ False

**Question 10** (1 point) ✓ *Saved*

In a SLAM system, a "Loop Closure" creates a new constraint (cost term) between the robot's current pose and a previously observed landmark. Select the true statement below:

- ☐ Adding the new cost term will **increase** the overall uncertainty (covariance) of the map as more noisy data is being added to the system.
- ☒ Adding the cost term will **decrease** the overall uncertainty (covariance) of the map as more noisy data is being added to the system.
- ☐ Adding the new cost term will **not change** the overall uncertainty (covariance) of the map.

## Quiz 9: SLAM

**Question 1** (1 point) ✓ *Saved*

In the context of SLAM, why is it often stated that "Localization and Mapping are co-dependent"?

- ☐ Because the Information Matrix requires an equal number of poses and landmarks.
- ☐ Because the robot cannot move unless both a map and a localization estimate are provided by the user.
- ☒ Because to build an accurate map, you need to know where you are; but to know where you are, you need an accurate map.
- ☐ Because sensors like LiDAR require a map to function, while encoders do not.

**Question 2** (1 point) ✓ *Saved*

In a Factor Graph represented as a mass-spring system, if we have a sensor measurement with very high uncertainty (large covariance), how is this represented physically?

- ☐ An increase in the length of the spring.
- ☐ A much stiffer spring for the measurement factor (higher spring constant).
- ☐ A heavier mass for the landmark node.
- ☒ A much looser spring for the measurement factor (lower spring constant).

**Question 3** (1 point) ✓ *Saved*

What is the primary reason the Information Matrix in SLAM is "Sparse" (containing many zeros)?

- ☒ The robot only observes a small subset of landmarks from any single pose, meaning most poses and landmarks have no direct constraint between them.
- ☐ Most landmarks are never seen by any robot pose.
- ☐ We manually optimize the entries in the matrix to make the code run faster.
- ☐ The robot's trajectory is always a straight line.

**QUESTION 4**

Whether the system employs a Filter or a Smoother determines how an error is fixed when a robot finishes a loop and finds a 2-meter discrepancy through loop closure.

### 1. The Extended Kalman Filter for filtering

By design, standard EKF's are forgetful. The present state ( $x_t, y_t, t$ ) is only estimated. When the robot recognizes the starting landmark, the EKF finds a noteworthy innovation (the 2-meter gap). The Kalman Gain is used to bring the robot's current position back to the landmark. The EKF has no effect on the trajectory's midway. Since the filter has already excluded the earlier postures and covariances, it cannot go back in time to fix them. Consequently, the trajectory will appear as a drifting, smooth arc with a sudden teleportation jump of two meters at the very end.

### 2. Batch Optimization (Graph-SLAM/Smoothing)

A Batch Optimizer (like GTSAM or g2o) manages the robot's whole history as a Factor Graph. Each pose ( $x_0, x_1, \dots, x_t$ ) is represented as nodes connected by springs (odometry factors). The

Update: By adding a loop closure factor, the final position ( $x_t$ ) and the beginning pose ( $x_0$ ) are connected by a stiff spring. The optimizer then performs a global minimization (like Levenberg-Marquardt) to find the configuration that simultaneously lowers each spring's tension. In the center of the trajectory, the 2-meter error is distributed backward throughout the entire circle. The optimizer bends the intermediate postures to absorb the error. Positions recorded halfway around the circle will be changed (for example, by about one meter) to ensure that the entire path is smooth and mathematically consistent. The leap is eliminated in favour of a slightly altered, more accurate overall shape.

| S                     | EKF (Filtering)                            | Batch Optimizer (Smoothing)                  |
|-----------------------|--------------------------------------------|----------------------------------------------|
| <b>Data Retention</b> | Keeps only the <i>current</i> pose.        | Keeps the <i>entire history</i> of poses.    |
| <b>Loop Closure</b>   | Snaps the current position (Point Update). | Re-optimizes the whole path (Global Update). |
| <b>Middle Poses</b>   | <b>Remain unchanged</b> (and incorrect).   | <b>Adjusted/Corrected</b> to fit the loop.   |

|                    |                         |                                      |
|--------------------|-------------------------|--------------------------------------|
| <b>Computation</b> | Fast and constant time. | Increases as the map grows (Sparse). |
|--------------------|-------------------------|--------------------------------------|

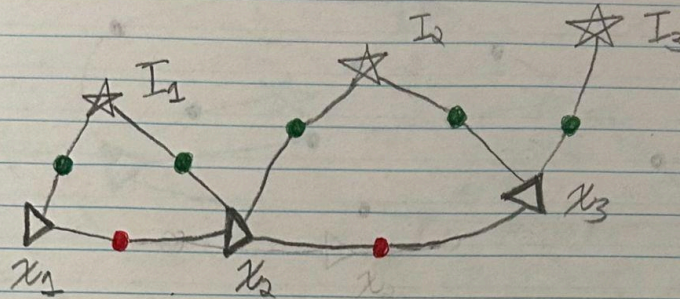
#### QUESTION 5

# QUIZ 9: SLAM

Page:

1

• Question 5



I) Square Information Matrix (A)

$$A = \begin{matrix} & \delta x_1 & \delta x_2 & \delta x_3 & \delta I_1 & \delta I_2 & \delta I_3 \\ \begin{matrix} \delta x_1 \\ \delta x_2 \\ \delta x_3 \\ \delta I_1 \\ \delta I_2 \\ \delta I_3 \end{matrix} & \begin{bmatrix} \text{shaded} & & & & & \\ \text{shaded} & \text{shaded} & & & & \\ & \text{shaded} & \text{shaded} & & & \\ \hline & & & \text{shaded} & & \\ & & & & \text{shaded} & \\ & & & & & \text{shaded} \end{bmatrix} \end{matrix}$$

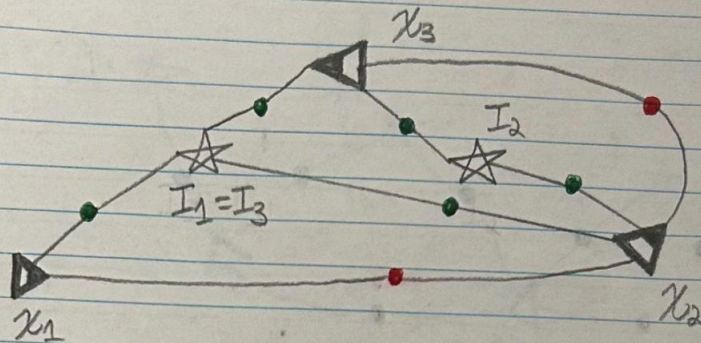
$$A = \begin{matrix} & \delta x_1 & \delta x_2 & \delta x_3 & \delta I_1 & \delta I_2 & \delta I_3 \\ \begin{matrix} \delta x_1 \\ \delta x_2 \\ \delta x_3 \\ \delta I_1 \\ \delta I_2 \\ \delta I_3 \end{matrix} & \begin{bmatrix} \text{shaded} & & & & & \\ \text{shaded} & \text{shaded} & & & & \\ & \text{shaded} & \text{shaded} & & & \\ \hline & & & \text{shaded} & & \\ & & & & \text{shaded} & \\ & & & & & \text{shaded} \end{bmatrix} \end{matrix}$$

$$A = \begin{matrix} & \delta x_1 & \delta x_2 & \delta x_3 & \delta I_1 & \delta I_2 & \delta I_3 \\ \begin{matrix} \delta x_1 \\ \delta x_2 \\ \delta x_3 \\ \delta I_1 \\ \delta I_2 \\ \delta I_3 \end{matrix} & \begin{bmatrix} \text{shaded} & \text{shaded} & 0 & \text{shaded} & 0 & 0 \\ \text{shaded} & \text{shaded} & 0 & \text{shaded} & 0 & 0 \\ 0 & \text{shaded} & \text{shaded} & 0 & \text{shaded} & \text{shaded} \\ \hline & & 0 & \text{shaded} & 0 & 0 \\ 0 & & & \text{shaded} & 0 & 0 \\ 0 & 0 & & & \text{shaded} & \text{shaded} \end{bmatrix} \end{matrix}$$

$$\delta x = \begin{bmatrix} \delta x_1 \\ \delta x_2 \\ \delta x_3 \\ \delta I_1 \\ \delta I_2 \\ \delta I_3 \end{bmatrix}$$

(2)

II)



• Square Information Matrix (A)

A =

| $\delta x_1$ | $\delta x_2$ | $\delta x_3$ | $\delta I_1$ | $\delta I_2$ |              |
|--------------|--------------|--------------|--------------|--------------|--------------|
|              |              |              |              |              | $\delta x_1$ |
|              |              |              |              |              | $\delta x_2$ |
|              |              |              |              |              | $\delta x_3$ |
|              |              |              |              |              | $\delta I_1$ |
|              |              |              |              |              | $\delta I_2$ |

A =

| $\delta x_1$ | $\delta x_2$ | $\delta x_3$ | $\delta I_1$ | $\delta I_2$ |              |
|--------------|--------------|--------------|--------------|--------------|--------------|
|              |              |              |              |              | $\delta x_1$ |
|              |              |              |              |              | $\delta x_2$ |
|              |              |              |              |              | $\delta x_3$ |
|              |              |              |              |              | $\delta I_1$ |
|              |              |              |              |              | $\delta I_2$ |

A =

| $\delta x_1$ | $\delta x_2$ | $\delta x_3$ | $\delta I_1$ | $\delta I_2$ |              |
|--------------|--------------|--------------|--------------|--------------|--------------|
|              |              |              |              |              | $\delta x_1$ |
|              |              |              |              |              | $\delta x_2$ |
|              |              |              |              |              | $\delta x_3$ |
|              |              |              |              |              | $\delta I_1$ |
|              |              |              |              |              | $\delta I_2$ |

$$\delta x = \begin{bmatrix} \delta x_1 \\ \delta x_2 \\ \delta x_3 \\ \delta I_1 \\ \delta I_2 \end{bmatrix}$$

When the robot realizes  $l_3$  and  $l_1$  are the same physical object, the two separate landmark nodes in the graph are collapsed/merged into a single node ( $l_1$ ). The measurement factor that previously connected  $x_3$  to  $l_3$  now connects  $x_3$  to  $l_1$ .

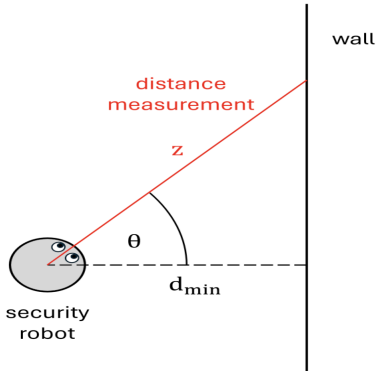
### Changes to the Matrix Structure:

1. The matrix shrinks from (6)(6) blocks to (5)(5) blocks because  $l_3$  is removed from the state vector.
2. A new non-zero block appears at  $(x_3, l_1)$  and its symmetric counterpart  $(l_1, x_3)$ .
3. The new order is  $[x_1, x_2, x_3, l_1, l_2]$ .

The structure and sparsity of the square information matrix  $A$  is impacted as the matrix becomes slightly less sparse relative to its size. By connecting  $x_3$  to  $l_1$ , a long-range constraint has been included. While the total number of zeros might still be high, we have increased the connectivity of  $l_1$ . This is a Loop Closure. Physically, it acts like a stiff spring connecting the end of the trajectory back to the beginning. When the batch optimizer solves this new matrix, the 2-meter drift accumulated between  $x_1$  and  $x_3$  will be distributed across the entire path, snapping the map into a globally consistent state.

## Quiz 8: Extended Kalman Filter

### QUESTION 1

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p>The diagram shows a security robot (a grey circle with an eye) positioned at a distance <math>d_{\min}</math> from a vertical wall. A horizontal dashed line extends from the robot to the wall. A red line labeled 'distance measurement' and <math>z</math> extends from the robot to the wall at an angle <math>\theta</math> from the horizontal. The wall is represented by a vertical line on the right.</p> | <p><b>Prior Belief</b></p> <p>Mean: <math>\bar{\theta}_p = 45^\circ = 0.785 \text{ rad}</math></p> <p>Uncertainty: <math>C_p = 0.05 \text{ rad}^2</math></p>                                                                                                                                                                                                                                                                                                |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <p><b>Observation</b></p> <p>Measurement Mean: <math>\bar{z} = 1.6\text{m}</math></p> <p>Measurement Uncertainty: <math>R = 0.05 \text{ m}^2</math></p> <p>Observation Model: <math>z = h(\theta) = \frac{d_{\min}}{\cos(\theta)} = \frac{1\text{m}}{\cos(\theta)}</math></p> <p>Observation Jacobian: <math>H = \frac{dh(\theta)}{d\theta} = \frac{d_{\min} \sin(\theta)}{\cos^2(\theta)} = \frac{1\text{m} \cdot \sin(\theta)}{\cos^2(\theta)}</math></p> |

### Questions

1. Using the observation model, what is the expected distance measurement given the robot's current prior belief? (1 point)

### 1. Expected Distance Measurement

Our prior mean is passed through the observation model  $h(\theta)$  to produce the expected measurement. Given:  $\bar{\theta} = 45^\circ$  and  $d_{min} = 1m$ .

$$h(\bar{\theta}_p) = \frac{d_{min}}{\cos(45^\circ)} = \frac{1}{(\frac{\sqrt{2}}{2})} = \sqrt{2} \approx 1.414m$$

### 2. Evaluate the observation Jacobian at the current prior belief. (1 point)

Evaluate the Jacobian  $H$

At the preceding mean  $\bar{\theta}_p = 45^\circ$  (or  $\frac{\pi}{4}$ ). Using the following formula:  $H = \frac{1m \cdot \sin(\theta)}{\cos^2(\theta)}$

$$H = \frac{\sin(45^\circ)}{\cos^2(45^\circ)} = \frac{(\frac{\sqrt{2}}{2})}{(\frac{\sqrt{2}}{2})^2} = \frac{(\frac{\sqrt{2}}{2})}{(\frac{1}{2})} = \sqrt{2} \approx 1.414$$

### 3. Given the noisy measurement and the prior belief, compute the Kalman gain $K$ for the correction step. (2 points)

Determine the Kalman Gain ( $K$ )

For a 1D state, the Kalman Gain is computed as follows:  $K = \frac{C_p H^T}{H C_p H^T + R}$

Given  $C_p = 0.05$ ,  $R = 0.05$ , and  $H = \sqrt{2}$

$$K = \frac{0.05 \cdot \sqrt{2}}{(\sqrt{2})^2 \cdot 0.05 + 0.05} = \frac{0.05 \cdot \sqrt{2}}{2 \cdot 0.05 + 0.05} = \frac{\sqrt{2}}{3} \approx 0.471$$

### 4. Calculate the corrected mean of the robot's orientation $\bar{\theta}$ (1 point)

Compute the Corrected Mean ( $\bar{\theta}_{corrected}$ ).

The innovation (the discrepancy between the actual measurement  $\bar{z}$  and the expected value) is incorporated into the corrected mean. Given:  $\bar{z} = 1.6m$ .

$$\theta_{corr} = \bar{\theta}_p + K(\bar{z} - h(\bar{\theta}_p))$$

$$\theta_{corr} = 0.785 + 0.471(1.6 - 1.414)$$

$$\theta_{corr} = 0.785 + 0.471(1.6 - 1.414) \approx 0.785 + 0.0867 = 0.8726 \text{ rad}$$

(This is roughly  $50.0^\circ$  when converted to degrees.)

## 5. Calculate the corrected covariance of the robot's orientation (1 point)

Determine the Corrected Covariance ( $C_{corrected}$ ).

Based on the data obtained from the sensor, the uncertainty update lowers the previous covariance.

$$C_{corr} = (1 - KH)C_p$$

$$C_{corr} = (1 - (0.471)(1.414))(0.05)$$

$$C_{corr} = (1 - 0.666)(0.05) = 0.334(0.05) = 0.0167 \text{ rad}^2$$

| Parameter            | Value             |
|----------------------|-------------------|
| Expected Measurement | \$1.414\text{m}\$ |
| Jacobian \$H\$       | \$1.414\$         |

|                      |                                    |
|----------------------|------------------------------------|
| Kalman Gain $K$      | $0.471$                            |
| Corrected Mean       | $0.873 \text{ rad}$ ( $50^\circ$ ) |
| Corrected Covariance | $0.0167 \text{ rad}^2$             |

Observe that the corrected orientation ( $50^\circ$ ) lies between the orientation suggested by the further-than-expected measurement ( $1.6 \text{ m}$  vs.  $1.41 \text{ m}$ ) and the prior belief ( $45^\circ$ ). Furthermore, the uncertainty decreased from  $0.05$  to  $0.0167$ , demonstrating the robot's increased confidence in its orientation.

**Question 2** (1 point) ✓ *Saved*

In an EKF, why do we need the Jacobian  $H$  instead of just using the observation model  $h(\theta)$  directly in the covariance update?

- ☒ To convert the non-linear observation model into a local linear approximation.
- ☐ Because  $h(\theta)$  only works for integers.
- ☐ To eliminate the sensor noise  $R$ .

**Question 3** (1 point) ✓ *Saved*

In the example above, if we changed the sensor to a much higher-quality one ( $R=0.001$ ), what would happen to the Kalman Gain  $K$ ?

- ☒ It would increase, putting more "trust" into the measurement.
- ☐ It would remain unchanged because the prior uncertainty stayed the same.
- ☐ It would decrease toward zero, putting more "trust" into the measurement.

Look at your result for the **posterior uncertainty** in the question above. Is it possible for a Kalman Filter update to result in a **posterior uncertainty that is *larger* than the prior uncertainty** you started with? Explain why or why not.

To answer the question of whether a Kalman Filter update may produce a posterior uncertainty greater than the prior uncertainty is, no. The update stage in a mathematically consistent Kalman Filter is especially made to add fresh data to improve an existing estimate. The posterior variance will always be less than, or in the extreme case of a wholly worthless sensor, equal to, the prior variance since information, even from a noisy or low-quality sensor, serves to narrow the range of possible states the robot could be in.

The computation of the Kalman Gain ( $K$ ) is what drives this decrease in uncertainty. In essence, the filter handles the update as a weighted average between the exterior measurement and the internal prediction, with weights based on the respective certainties of both. This is evident from the covariance's mathematical update,  $P_K^+ = (I - KH)P_K^-$ , where the term  $(I - KH)$  functions as a fractional multiplier that shrinks the prior covariance ( $P_K^-$ ) in accordance with the information gain. The filter only gives a sensor a very low weight ( $K \approx 0$ ), which results in a tiny but non-negative reduction in uncertainty even if the sensor is much noisier than the robot's internal model.

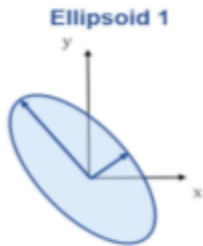
Differentiating between the update and prediction steps is crucial. Only when physical stochasticity, such as wheel slip or motor noise, is introduced into the system during the prediction (motion) phase does uncertainty rise. Although linearization mistakes or extreme outliers can occasionally cause real-world Extended Kalman Filter (EKF) implementations to diverge, these are regarded as model approximation failures rather than a characteristic of the Kalman Filter logic itself. Observing the world always results in greater clarity, never less, under the assumptions of Gaussian noise and an accurate model.

## Quiz 7

### Question 1 (2 points) ✓ Saved

Covariance 1

$$\mathbf{C} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$



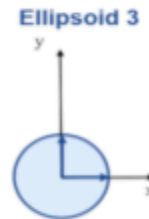
Covariance 2

$$\mathbf{C} = \begin{bmatrix} 0.1 & 0 \\ 0 & 5 \end{bmatrix}$$



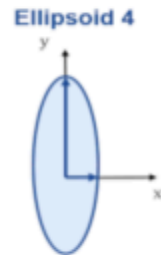
Covariance 3

$$\mathbf{C} = \begin{bmatrix} 2.5 & 0 \\ 0 & 0.2 \end{bmatrix}$$



Covariance 4

$$\mathbf{C} = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$$



Match the following 2D Covariance Matrices  $\mathbf{C}$  to their corresponding uncertainty ellipsoids.

- 1  Covariance 4
- 4  Covariance 2
- 3  Covariance 1
- 2/4  Covariance 3

1. Ellipsoid 1
2. Ellipsoid 2
3. Ellipsoid 3
4. Ellipsoid 4

### Question 3 (1 point) ✓ Saved

Assume we are commanding a differential-drive mobile robot to drive in a straight line by commanding identical velocities to both of its wheels. In this case, the resulting uncertainty distribution over its position (x,y) in the global frame will always remain a perfect circle. True or False?

- ☐ True
- ☒ False

Question 2 (2 points) ✓ Saved

In class, we derived the following fomulation to propagate the uncertainty through the motion model for mobile robots

$$\mathbf{C}_{k+1} = \mathbf{G}_x \mathbf{C}_k \mathbf{G}_x^T + \mathbf{G}_u \mathbf{C}_\varphi \mathbf{G}_u^T$$

In your own words, explain what this uncertainty update is composed of. Explain the physical meaning of both components.

## Accounting for Uncertain Inputs and Last State

- We can now compute the approximated Gaussian of our new state
- The new mean is

$$\bar{\mathbf{x}}(t_{k+1}) = \mathbf{g}(\bar{\mathbf{x}}(t_k), \bar{\mathbf{u}})$$

- The new covariance is

$$\mathbf{C}_{x_{k+1}} = \mathbf{G}_x \mathbf{C}_{x_k} \mathbf{G}_x^T + \mathbf{G}_u \mathbf{C}_\varphi \mathbf{G}_u^T$$

old uncertainty carried forward through motion model

additional uncertainty from noisy motion process

- The total expression is

$$\mathbf{x}(t_{k+1}) \sim \mathcal{N}(\mathbf{g}(\bar{\mathbf{x}}(t_k), \bar{\mathbf{u}}), \mathbf{G}_x \mathbf{C}_{x_k} \mathbf{G}_x^T + \mathbf{G}_u \mathbf{C}_\varphi \mathbf{G}_u^T)$$

- We can see that the uncertainty will now grow, as we iteratively update the robot's state

This formula is the typical Uncertainty Propagation step in EKF (Extended Kalman Filter) localization. It explains how, as the robot advances, its confusion about its position increases. Thus, meaning that the resulting distribution of the robot's state is Gaussian in the Euclidean space. The motion uncertainty is determined using linear approximations in this formula as the distribution of the robot's state with a Gaussian by linearizing the motion propagation. In essence, the total uncertainty at the subsequent time step (CK+1) is equal to the sum of our previous ignorance and our recent error.

- $\mathbf{G}_x \mathbf{C}_k \mathbf{G}_x^T$ : Changed Prior Uncertainty

The prior uncertainty, or where it was believed that at the previous time step, is represented by CK. The State Jacobian, or  $\mathbf{G}_x$ , shows how a slight alteration in the past state influences the present state. Therefore, essentially meaning that the old uncertainty is carried forward through the motion model.

This phrase explains why, if you were unsure about your location and direction just a moment ago, that confusion spreads and frequently grows depending on your kinematics. For

instance, if you were unsure of your heading ( $\theta$ ) at step K, you would have driven farther away from your planned (x,y) coordinate at that step K+1 due to that heading inaccuracy. To account for this movement, the  $G_x$  matrices stretch your previous uncertainty.

- GuCøGuT: New Error (Additional Process Noise)

The covariance of the input noise (e.g., wheel slip, motor jitter) is C (often represented as Q). The Input Jacobian, or  $G_u$ , maps the relationship between mistakes in your controls (the wheels) and faults in your global position. Therefore, essentially accounting for the additional uncertainty from the noisy motion process.

This is the new uncertainty brought forth by the current trend; because hardware is not perfect, this term would still exist even if you knew exactly where you started ( $CK=0$ ). As an illustration, consider a robot that rotates a little more slowly than instructed and has a noisy left wheel. This term determines how the robot is pushed off its intended route at this particular step by that particular hardware noise (C).

| Term               | Mathematical Name | Physical Analogy                                            |
|--------------------|-------------------|-------------------------------------------------------------|
| $G_x C_K G_x^T$    | State Prediction  | Carrying over and magnifying your old mistakes.             |
| $G_u C_\phi G_u^T$ | Process Noise     | Adding new mistakes caused by the environment and hardware. |

#### Question 4 (5 points)

A robot's one-dimensional position is updated using a model where the control effect is proportional to the square of its current position:

$$x_{k+1} = g(x_k, u_k) = x_k + u_k \cdot x_k^2$$

Assume the following initial conditions for the state:

$$\begin{aligned} \text{Initial mean: } \bar{x}_0 &= 1\text{m} \\ \text{Initial variance: } \sigma_{x_0}^2 &= 0 \end{aligned}$$

Additionally, consider the following control input with constant mean and variance:

$$\begin{aligned} \text{Control input: } \bar{u} &= 1 \\ \text{Control variance: } \sigma_u^2 &= 0.5 \end{aligned}$$

**Task:** Calculate the mean and variance for the next two steps ( $k = 1, k = 2$ ). Show the calculations of your state and input derivatives  $G_x$  and  $G_u$ .

## What Happens if our Last State is also Noisy?

- Assume both our wheel velocities and our last state are uncertain

$$\mathbf{u} = \begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \bar{\phi}_1 \\ \bar{\phi}_2 \end{bmatrix}, \mathbf{C}_\phi\right), \quad \mathbf{x}(t_k) = \begin{bmatrix} x(t_k) \\ y(t_k) \\ \theta(t_k) \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \bar{x}(t_k) \\ \bar{y}(t_k) \\ \bar{\theta}(t_k) \end{bmatrix}, \mathbf{C}_{x_k}\right), \quad \mathbf{x}(t_{k+1}) = \mathbf{g}(\mathbf{x}(t_k), \mathbf{u}) = \begin{bmatrix} x(t_k) + \cos(\theta(t_k)) \frac{r}{2} (\dot{\phi}_1 + \dot{\phi}_2) \cdot \Delta T \\ y(t_k) + \sin(\theta(t_k)) \frac{r}{2} (\dot{\phi}_1 + \dot{\phi}_2) \cdot \Delta T \\ \theta(t_k) + \frac{r}{2b} (\dot{\phi}_1 - \dot{\phi}_2) \cdot \Delta T \end{bmatrix}$$

- We can follow the same procedure as before, but need to propagate the uncertainty of both uncertain variables  $\mathbf{u}$  and  $\mathbf{x}(t_k)$
- That means we have to linearize our function  $\mathbf{g}(\mathbf{x}(t_k), \mathbf{u})$  with respect to both the inputs  $\mathbf{u}$  and the last state  $\mathbf{x}(t_k)$

$$\mathbf{G}_u = \frac{\partial \mathbf{x}(t_{k+1})}{\partial \mathbf{u}} = \frac{\partial \mathbf{x}(t_{k+1})}{\partial (\dot{\phi}_1, \dot{\phi}_2)}, \quad \mathbf{G}_x = \frac{\partial \mathbf{x}(t_{k+1})}{\partial \mathbf{x}(t_k)} = \frac{\partial \mathbf{x}(t_{k+1})}{\partial (x(t_k), y(t_k), \theta(t_k))}$$

# Motion Model Propagation Linearization

- Motion model propagation given as

$$\mathbf{x}(t_{k+1}) = \mathbf{g}(\mathbf{x}(t_k), \mathbf{u}) = \begin{bmatrix} x(t_k) + \cos(\theta(t_k)) \frac{r}{2} (\dot{\phi}_1 + \dot{\phi}_2) \cdot \Delta T \\ y(t_k) + \sin(\theta(t_k)) \frac{r}{2} (\dot{\phi}_1 + \dot{\phi}_2) \cdot \Delta T \\ \theta(t_k) + \frac{r}{2b} (\dot{\phi}_1 - \dot{\phi}_2) \cdot \Delta T \end{bmatrix}$$

- Taking the partial derivatives yields

$$\mathbf{G}_u = \frac{\partial \mathbf{x}(t_{k+1})}{\partial (\dot{\phi}_1, \dot{\phi}_2)} = \begin{bmatrix} \frac{r}{2} \cos(\theta(t_k)) \cdot \Delta T & \frac{r}{2} \cos(\theta(t_k)) \cdot \Delta T \\ \frac{r}{2} \sin(\theta(t_k)) \cdot \Delta T & \frac{r}{2} \sin(\theta(t_k)) \cdot \Delta T \\ \frac{r}{2b} \cdot \Delta T & -\frac{r}{2b} \cdot \Delta T \end{bmatrix}, \quad \mathbf{G}_x = \frac{\partial \mathbf{x}(t_{k+1})}{\partial (x(t_k), y(t_k), \theta(t_k))} = \begin{bmatrix} 1 & 0 & -\sin(\theta(t_k)) \frac{r}{2} (\dot{\phi}_1 + \dot{\phi}_2) \cdot \Delta T \\ 0 & 1 & \cos(\theta(t_k)) \frac{r}{2} (\dot{\phi}_1 + \dot{\phi}_2) \cdot \Delta T \\ 0 & 0 & 1 \end{bmatrix}$$

$\frac{\partial \mathbf{x}(t_{k+1})}{\partial x(t_k)}$        $\frac{\partial \mathbf{x}(t_{k+1})}{\partial y(t_k)}$        $\frac{\partial \mathbf{x}(t_{k+1})}{\partial \theta(t_k)}$

## Accounting for Uncertain Inputs and Last State

- We can now compute the approximated Gaussian of our new state
- The new mean is

$$\bar{\mathbf{x}}(t_{k+1}) = \mathbf{g}(\bar{\mathbf{x}}(t_k), \bar{\mathbf{u}})$$

- The new covariance is

$$\mathbf{C}_{\mathbf{x}_{k+1}} = \mathbf{G}_x \mathbf{C}_{\mathbf{x}_k} \mathbf{G}_x^T + \mathbf{G}_u \mathbf{C}_\varphi \mathbf{G}_u^T$$

old uncertainty carried forward  
through motion model  
 additional uncertainty from noisy  
motion process

- The total expression is

$$\mathbf{x}(t_{k+1}) \sim \mathcal{N}(\mathbf{g}(\bar{\mathbf{x}}(t_k), \bar{\mathbf{u}}), \mathbf{G}_x \mathbf{C}_{\mathbf{x}_k} \mathbf{G}_x^T + \mathbf{G}_u \mathbf{C}_\varphi \mathbf{G}_u^T)$$

- We can see that the uncertainty will now grow, as we iteratively update the robot's state

To solve this, we will use the EKF propagation equations for mean ( $\bar{x}$ ) and variance ( $\sigma^2$ ). Since this is a 1D problem, the Jacobians  $G_x$  and  $G_u$  simplify to scalar partial derivatives.

## 1. Deriving the Jacobians

Given the motion model:  $x_{k+1} = g(x_k, u_k) = x_k + u_k \cdot x_k^2$

- **State Derivative ( $G_x$ ):**

$$\frac{\partial g}{\partial x_k} = 1 + 2(u_k \cdot x_k)$$

- **Input Derivative ( $G_u$ ):**

$$\frac{\partial g}{\partial u_k} = x_k^2$$

## 2. Step k=1 (Propagation from k=0)

**Initial State:**  $\bar{x}_0 = 1, \sigma_{x_0}^2 = 0$

**Control:**  $\bar{u} = 1, \sigma_u^2 = 0.5$

- **Mean Update:**

$$\bar{x}_1 = \bar{x}_0 + \bar{u} \cdot \bar{x}_0^2 = 1 + 1(1)^2 = 2m$$

- **Jacobian Evaluation (at k=0):**

$$G_x = 1 + 2(1)(1) = 3$$

$$G_u = (1)^2 = 1$$

- **Variance Update:**

$$\sigma_{x_1}^2 = G_x^2 \sigma_{x_0}^2 + G_u^2 \sigma_u^2 = 3^2(0) + 1^2(0.5) = 0.5$$

## 3. Step k=2 (Propagation from k=1)

**State:**  $\bar{x}_1 = 2, \sigma_{x_1}^2 = 0.5$

**Control:**  $\bar{u} = 1, \sigma_u^2 = 0.5$

- **Mean Update:**

$$\bar{x}_2 = \bar{x}_1 + \bar{u} \cdot \bar{x}_1^2 = 2 + 1(2)^2 = 6m$$

- **Jacobian Evaluation (at k=1):**

$$G_x = 1 + 2(1)(2) = 5$$

$$G_u = (2)^2 = 4$$

- **Variance Update:**

$$\sigma_{x_2}^2 = G_x^2 \sigma_{x_1}^2 + G_u^2 \sigma_u^2 = 5^2(0.5) + 4^2(0.5) = 12.5 + 8 = 20.5$$

| Step ( $k$ ) | Mean ( $\bar{x}$ ) | Variance ( $\sigma_x^2$ ) |
|--------------|--------------------|---------------------------|
| 0            | 1m                 | 0                         |
| 1            | 2m                 | 0.5                       |
| 2            | 6m                 | 20.5                      |

Observe how the variance grows explosively in the second step. This is because the control noise  $\sigma_u^2$  is multiplied by  $G_u^2$  (which is  $x_k^4$ ), meaning as the robot gets further away, any small error in the wheels has a massive squared impact on the final position.

To solve this, we will use the EKF propagation equations for mean ( $\bar{x}$ ) and variance ( $\sigma^2$ ). Since this is a 1D problem, the Jacobians  $G_x$  and  $G_u$  simplify to scalar partial derivatives.

## Deriving the Jacobians

Given the motion model:  $x_{k+1} = g(x_k, u_k) = x_k + u_k \cdot x_k^2$

- **State Derivative ( $G_x$ ):**

$$\frac{\partial g}{\partial x_k} = 1 + 2(u_k \cdot x_k)$$

- **Input Derivative ( $G_u$ ):**

$$\frac{\partial g}{\partial u_k} = x_k^2$$

### 1. Step k=1 (Propagation from k=0)

**Initial State:**  $x_0=1$  ,  $\sigma_{x0}=0$

**Control:**  $u=1$ ,  $\sigma_u=0.5$

- **Mean Update:**  
 $x_1 = x_0 + u \cdot x_0 = 1 + 1(1) = 2\text{m}$
- **Jacobian Evaluation (at  $k=0$ ):**  
 $G_x = 1 + 2(1)(1) = 3$   
 $G_u = (1)^2 = 1$
- **Variance Update:**  
 $\sigma_{x1} = G_x^2 \sigma_{x0} + G_u^2 \sigma_u = 3^2(0) + 1^2(0.5) = 0.5$

## 2. Step $k=2$ (Propagation from $k=1$ )

**State:**  $x_1=2$ ,  $\sigma_{x1}=0.5$

**Control:**  $u=1$ ,  $\sigma_u=0.5$

- **Mean Update:**  
 $x_2 = x_1 + u \cdot x_1 = 2 + 1(2) = 6\text{m}$
- **Jacobian Evaluation (at  $k=1$ ):**  
 $G_x = 1 + 2(1)(2) = 5$   
 $G_u = (2)^2 = 4$
- **Variance Update:**  
 $\sigma_{x2} = G_x^2 \sigma_{x1} + G_u^2 \sigma_u = 5^2(0.5) + 4^2(0.5) = 12.5 + 8 = 20.5$

| Step (kkk) | Mean (xxx) | Variance ( $\sigma_x$ ) |
|------------|------------|-------------------------|
| 0          | 1m         | 0                       |
| 1          | 2m         | 0.5                     |
| 2          | 6m         | 20.5                    |

Observe how the variance grows explosively in the second step. This is because the control noise  $\sigma_u$  is multiplied by  $G_u^2$  (which is  $x_k^4$ ), meaning that as the robot gets further away, any small error in the wheels has a massive squared impact on the final position.

## Quiz 6

### Question 1 (2 points) ✓ Saved

In the context of a proportional (P) controller, what is the primary effect of increasing the proportional gain ( $K_p$ )?

- ☐ It increases the rise time and eliminates steady-state error.
- ☒ It decreases the rise time but may increase overshoot.
- ☐ It eliminates oscillations caused by sensor noise.
- ☐ It prevents the robot from overshooting the desired state.

### Question 3 (2 points) ✓ Saved

When implementing path-tracking for a differential-drive robot, what happens if the controller relies solely on heading control ( $e_H$ ) and ignores lateral error ( $e_L$ )?

- ☐ The robot will constantly overshoot the path.
- ☐ The robot will drive in endless circles.
- ☒ The robot will stay parallel to the path but will not necessarily move onto it.
- ☐ The robot will reach the path faster than with a combined controller.

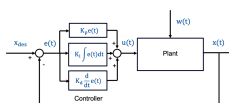
### Question 5 (1 point) ✓ Saved

Which component of a PID controller acts as a "damper" to reduce oscillations and overshooting the reference state?

- ☐ Proportional (P) Term
- ☐ Integral (I) Term
- ☒ Derivative (D) Term

#### PID-Controller Summary

| Term             | Rule                    | Effect on Rise Time | Effect on Overshoot | Effect on Steady-State Error |
|------------------|-------------------------|---------------------|---------------------|------------------------------|
| Proportional (P) | Drives toward reference | Decreases           | Increases           | Decreases                    |
| Integral (I)     | Eliminates steady bias  | Decreases           | Increases           | Eliminates                   |
| Derivative (D)   | Dampens oscillation     | Decreases           | Decreases           | No effect                    |



Explain the specific role of the Integral (I) term in a PI-controller. Why is it often necessary when a robot is operating in a real-world environment with disturbances like wheel slip or systematic model biases?

In PI (Proportional, Integral - Present, Past) controllers, the purpose of the integral (I) term (the "past") is to eliminate steady-state error, which refers to the difference between where the robot actually is and where it should be. A controller with proportional (P) only will not be able to address this issue. The I (Integral) term accumulates previous error values over time. The integration of the error through a signal leads to sustained growth of the I term value; therefore, until zero error is established, regardless of the size, the I term will increment indefinitely.

The resulting variable acts to ensure that from this increased I term value, the controller provides a greater than expected output signal to the system until it ultimately closes the gap and reaches the desired location.

Although a proportional-only controller with high proportional gain will theoretically work in a controlled mathematical/ideal environment, real-world robots are constantly faced with real-world constraints that require the use of an integral (I) term.

- **Systematic Model Bias:** Physical imperfections are often present in robots, such as one wheel being larger than the other or a motor that requires a minimum voltage to overcome its internal resistance. A Proportional controller may reach a point where the error is so small that the resulting output is unable to overcome this internal resistance, resulting in the robot never reaching its desired location.
- **Wheel Slip:** External forces, such as a constant wind blowing sideways, a slope in the floor, or consistent wheel slip on different surfaces, can be classified as biases. The Integral term learns that the current control effort is not sufficient to counter these forces and increases the power output until the disturbance has been countered and the error equals zero.
- **Real-World Environmental Disturbances:** The I-term can be a significant contributor to the control of the robot, but it can also contribute to the I-term windup issue. When the robot's actuators reach the physical maximum (saturation), the I-term continues to accumulate a value that cannot be executed as immediate output (massive value); this often results in a violently overshooting or unstable situation.

**Question 4** (3 points) ✓ *Saved*

A student implements the following path-tracking control law for a differential-drive robot:

$$\omega(t) = -K_{p,H}e_H(t) - K_{p,L}e_L(t)$$

During testing, they observe that the robot oscillates wildly back and forth across the reference path without ever settling down.

**Questions**

1. Which of the two gains is likely too high relative to the other?
  2. Using the concepts from the lecture and tutorial, explain why this overshooting behavior is happening and how it can be overcome.
- Based on the control law and the principles of path tracking for differential-drive robots, the analysis of the oscillating robot suggests that the Lateral Gain ( $K_{p,L}$ ) is likely too high relative to the Heading Gain ( $K_{p,H}$ ) since the robot oscillates wildly back and forth across the reference path without ever settling down.
  - The control laws provided are proportional (P) controllers of the lateral error  $e_L(t)$  and the heading error  $e_H(t)$ . In this case, the typical behaviour of an under-damped system (oscillating wildly) is evidenced.  
The lateral error indicates how far away the robot is from the path of travel, while the heading error indicates how the robot should adjust its heading to be aligned with the path.  
If  $K_{p,L}$  is too aggressive in closing the distance to the path, the robot will quickly turn towards the path and will have no means of damping or counter-steering to allow the robot to straighten out as it approaches the line. The combination of the robot's inertia and the very high rate of turn results in being projected right across the reference line. When the robot crosses to the other side of the reference line, a large amount of error will be generated that will be in the opposite direction to the previous reference point, resulting in yet another very sharp turn. Because there is no way to slow the correction as the error approaches zero, the robot will have sustained oscillations.  
To stabilize the robot and cease the robot's oscillation (overcoming the overshooting behaviour, the student can implement the following strategies from the lecture and tutorial:
    - The easiest way would be to rebalance or increase the percentage of gain between  $K_{p,L}$  and  $K_{p,H}$ . If you increase the heading gain  $K_{p,H}$ , this can be interpreted as an increase in "stiffness" of the robot and forces the robot to be more focused on remaining parallel to the path. By doing so there is a natural

resistive force to the sharp oscillations of the robot created by the lateral corrections.

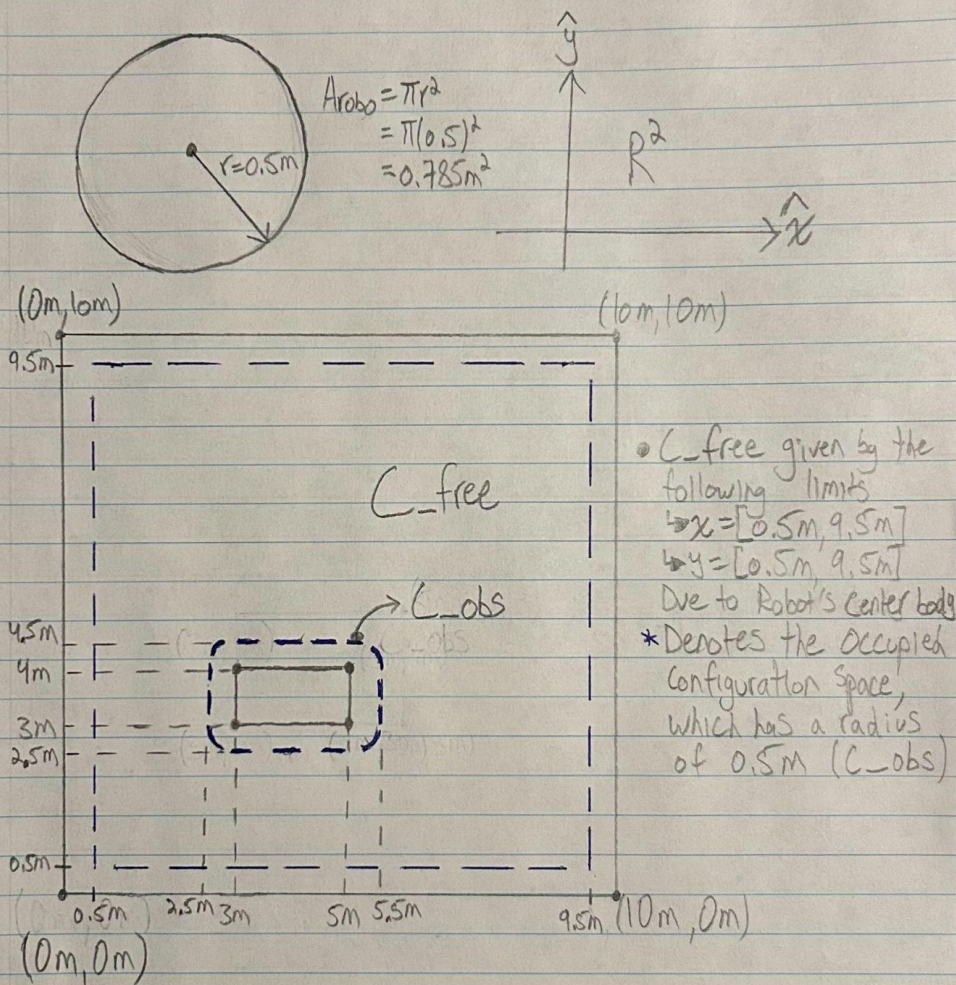
- The tutorial also suggested that instead of finding the error relative to the current closest point where the robot is located, you can steer the robot toward a point further ahead along the path (look-ahead point) so that the robot's steering commands are smoothed out, and the tendency to over-correct for lateral displacement is significantly reduced.
- In PID control, a common solution is to add a derivative component  $K_d(-1)\{\dot{e}\}_L$ . The D term acts as a damper to the angular acceleration experienced by the robot as it approaches the path rapidly. When the robot is at the closest point to the path, the D term applies a braking torque, providing damping of the robot's rotation and allowing a smooth transition onto the path.

# Quiz 5

## Q1

## Quiz 05: Path Planning

1.



2. Obstacle Area =  $(2\text{m})(1\text{m}) = 2\text{m}^2$

Side expansions:

Horizontal Sides (two) =  $2 [(2m)(0.5m)] = 2m^2$

■ Vertical Sides (two) =  $2[(9\text{m})(0.5\text{m})] = 1\text{m}^2$

Corner expansions:

• Given the radius of  $r = 0.5 \text{ m} \Rightarrow \text{Area} = \pi r^2 = \pi (0.5)^2 = 0.785 \text{ m}^2$

Total Area:  $C_{-obs} = \sum area = 2m^2 + 2m^2 + 1m^2 + 0.785m^2 = 5.785m^2$

$\therefore$  the total area occupied by Configuration Space is  $5.785 \text{ m}^2$

## Q2

1. Effective Width ( $W_{\{C_{free}\}}$ ) =  $\{\text{Physical Space}\} - 2 \times \{\text{Robot Radius}\}$

The robot's radius is multiplied by 2 as this provides the Diameter (in terms of width) for the circular robot. Therefore, the effective width of the doorway in the free configuration space is given by:

$$W_{\{C_{free}\}} = 1.0 \{m\} - 2(0.25\{m\}) = 1.0 \{m\} - 0.5 \{m\} = 0.5 \{m\}$$

2. With a  $\{C_{free}\}$  gap of 0.5 {m}, the resolution must be  $0.5\{m\}/\{2\} = 0.25 \{m\}$

To guarantee that a grid-based A\* planner can always find a path through a passage, the cell size ( $C_S$ ) must be small enough that at least one sequence of grid cells can fit entirely within the passage, regardless of the grid's translation or alignment.

To ensure that at least one whole cell always falls within a gap of width  $w$  (regardless of any arbitrary shift), the cell size ( $C_S$ ) must satisfy the following condition:  $C_S \leq w/2$ .

## Q3

1. No, the Manhattan distance is not an admissible heuristic for this problem.

For a heuristic to be admissible, it must never overestimate the actual cost to reach the goal ( $h(n) \leq h^*(n)$  for all  $n$ ).

In a 2D Euclidean space, the shortest possible distance between two points is a straight line (the Euclidean distance). The Manhattan distance ( $|x_n - x_{goal}| + |y_n - y_{goal}|$ ) is almost always longer than the straight-line Euclidean distance.

Since the edges in your graph represent the Euclidean distance between nodes, the real shortest path ( $h^*$ ) is already based on Euclidean geometry. Since the Manhattan distance can be larger than the Euclidean distance, it can overestimate the actual cost-to-go, violating the condition for admissibility. If the heuristic overestimates, A\* may settle on a suboptimal path because it wrongly assumes the optimal path is too expensive to explore.

2. An alternative heuristic that guarantees A\* will find the optimal path is the Euclidean distance.

$$h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$

The straight-line distance between two points is the absolute minimum distance possible in a Euclidean space. Since the robot is restricted to moving along specific edges (which might be jagged or indirect), the actual cost to reach the goal ( $h^*$ ) will always be greater than or equal to this straight-line distance. Therefore,  $h(n) \leq h^*(n)$  is guaranteed.

Furthermore, the Euclidean distance is a consistent (or monotonic) heuristic in this context. This means that for any two nodes  $n$  and  $m$  connected by an edge, the heuristic estimate at  $n$  is no greater than the cost of the edge to  $m$  plus the heuristic estimate at  $m$ .

#### Q4

In contrast to a basic RRT algorithm, where one connects new random samples to the nearest point from the previous sample to create an efficient pathway, RRT\* introduces an additional rewire phase that allows the sample to be rewired to more efficient parents based on all of the other samples within a given neighbourhood of that sample.

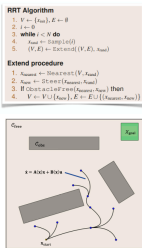
The RRT or Rapidly-exploring Random Tree, is concerned with searching only; once an additional node has been connected to the RRT, the way it was connected will not change, which results in many jagged or indirect paths to the end destination.

The RRT\* or Rapidly-exploring Random Tree\* functions differently; when connecting a new node to the tree, it will check to see if the new node can be reached at a lower total distance (from beginning to new node) than what is the present nearest node to that new node, and if so will rewire that new node to the present nearest node in the RRT for shortest distance. If it finds a node in the RRT that can be reached at a lower distance than that node, then the parent node of the RRT will be modified so that this new node will act as a parent node to all of its immediate child nodes.

The RRT\* algorithm is guaranteed to have improved **optimality** as compared to the generated path of the RRT in terms of the properties. While RRT can determine whether or not a path can be found, the paths produced by RRT are typically not very efficient, even if they do exist. As the number of iterations used to generate a path increases, RRT\* produces an output that converges on the most optimal (i.e., shortest) path.

## Rapidly-Exploring Random Trees (RRT)

- RRTs are used to intrinsically produce paths that are feasible by the motion model
- The procedure of RRTs is straightforward:
  - Randomly pick a point in space
  - Select the node in the tree closest to the point
  - Select a control input that would nominally move the robot towards the points
  - Simulate the path of the vehicle to determine the expected result
  - Repeat until the goal state is reached
- Sampling strategy involves picking random points in space, while picking points in the goal region from time to time
- This allows the tree to grow into most of the reachable space



## RRT\* Algorithm

- To address the limitations regarding optimality of RRT, RRT\* was proposed
- Two modifications w.r.t. conventional RRTs are made
- The algorithm is not terminated when the goal is reached
  - A second termination criteria is defined (e.g., maximum number of iterations, maximum tree size, maximum runtime)
  - When finished, the path with the shortest distance will be returned
  - Solution can improve as time goes by
- The nearest node in the tree is chosen so that each node is connected to the initial state using the shortest path
  - Several nodes in close proximity to the new sampled node may be rewired to ensure this

## RRT Properties

### Completeness

- **Probabilistic complete:** The probability that the RRT algorithm finds a feasible path approaches one, the longer the algorithm runs, if there exists one

### Optimality

- RRT is not optimal and will return non-optimal paths
- Paths often exhibit 'kinks' and 'detours'

### Efficiency

- RRT is considered highly efficient
- Scales up well to higher dimensions (unlike grid searches) and allows to handle kinematic constraints easily

## Properties of RRT\*

### Completeness

- **Probabilistic complete:** The probability that the RRT\* algorithm finds a feasible path approaches one, the longer the algorithm runs, if there exists one

### Optimality

- **Anytime optimal:** The longer the algorithm runs, the path found by RRT\* becomes better, slowly approaching the best path (asymptotic convergence)

### Efficiency

- RRT\* is still considered highly efficient, but is computationally 'heavier' than RRT due to introduced rewiring step
- Scales up well to higher dimensions (unlike grid searches) and allows to handle kinematic constraints easily

Q5

False

## A\* Algorithm

```

1: OPEN ← {1}
2: past_cost[1] ← 0, past_cost[node] ← infinity for node ∈ {2,...,N}
3: while OPEN is not empty do
4:   current ← first node in OPEN, remove from OPEN
5:   add current to CLOSED
6:   if current is in the goal set then
7:     return SUCCESS and the path to current
8:   end if
9:   for each nbr of current not in CLOSED do
10:    tentative_past_cost ← past_cost[current] + cost[current, nbr]
11:    if tentative_past_cost < past_cost[nbr] then
12:      past_cost[nbr] ← tentative_past_cost
13:      parent[nbr] ← current
14:      put (or move) nbr in sorted list OPEN according to
        past_total_cost[nbr] ← past_cost[nbr] +
        heuristic_cost_to_go(nbr)
15:   end if
16: end for
17: end while
18: return FAILURE

```

Keep open set of nodes, initialize with start node  
Initialize past cost to reach each node  
Run through each node in the open set (always pick first in set) and move to closed set  
If node is goal, we are done  
Run through all the neighbors of the current node, that are not in the closed set yet  
For each neighbor, compute the cost to reach this node  
Compute the estimated cost to reach the goal from here, given some heuristic  
Move neighbor node to the open set and order based on estimated cost

## Properties of A\*

### Completeness

- **Complete:** A\* complete planner either finds a feasible path or reports that there is none in finite time
- **Resolution complete:** A\* (if applied to a grid) is guaranteed to find a feasible path if the resolution of an underlying grid is fine enough and there exists one

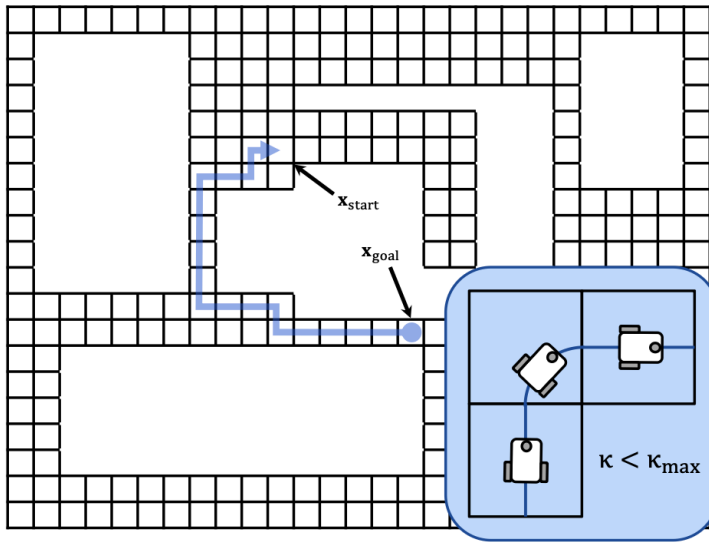
### Optimality

- **Optimal:** A\* always finds the best (i.e., lowest cost) path, if there exists one (given that the heuristic never overestimates the cost to reach the goal)

### Efficiency

- **Optimally efficient:** No other optimal algorithm, using the same heuristic information, is guaranteed to expand fewer nodes than A\*

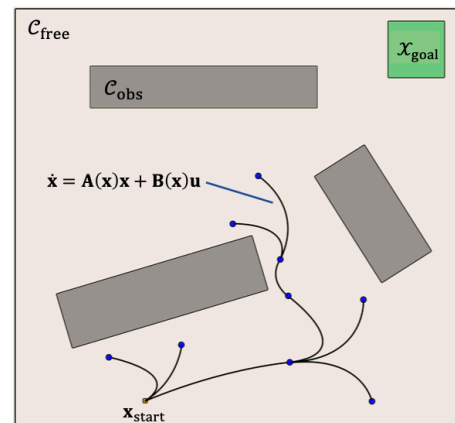
## Limitations of Grid-Based Planning



- **Scalability:** High-resolution grids drastically increase computational cost
- **Kinematic Infeasibility:** Paths are often jagged and ignore the vehicle's actual motion model (e.g., turning radius constraints)
- A more robust approach incorporates the robot's motion model directly into the planning search

## Planning with Vehicle Models

- Robots follow specific models (e.g., unicycle model for differential drive), moving along defined trajectories like circular arcs
- Paths during planning should satisfy the motion model constraints to be kinematically feasible
- Finding control inputs  $u(t)$  that drive  $x_{start}$  to a goal region  $x_{goal}$  while avoiding obstacles is mathematically challenging as it requires inverting differential equations
- Instead, we will discuss a popular numerical approach from the literature, which uses the robot's motion model to build a search tree expanding from  $x_{start}$  to explore the reachable space



# Quiz 4

## Q1

Automation refers to pre-programmed actions, while autonomy involves agency and decision-making.

## Q2

Level 3: Conditional Autonomy.

## Q3

The vehicle seeks out oxygen and stays close to it, avoids high temperatures, and aggressively seeks out light.

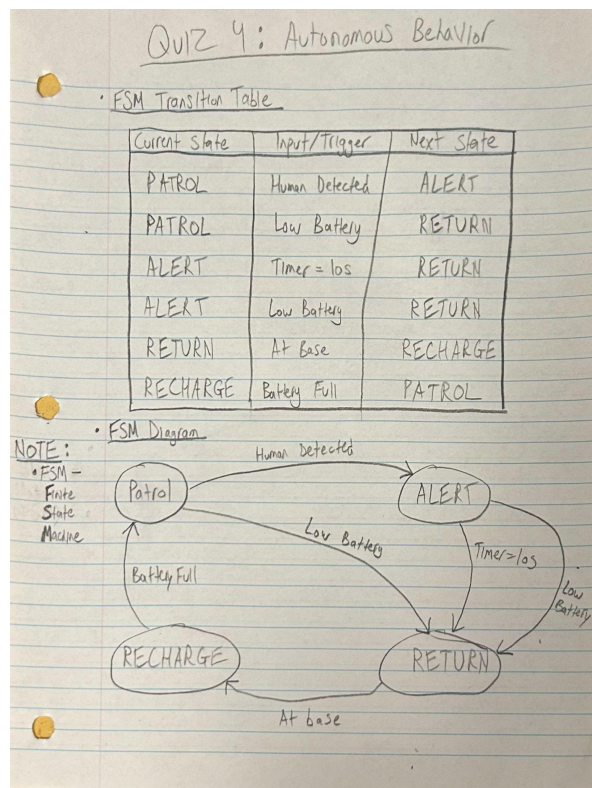
## Q4

Sense

Plan

Act

## Q5



## Quiz 3

### Q1

The two primary methods used by RGB-D cameras (Red, Green, Blue - Depth) to determine information are Structured Light and Time-of-Flight (ToF).

#### 1. Structured Light Mechanism:

This technique projects a predetermined pattern, either a grid or a pseudo-random dot pattern, onto a scene using an infrared (IR) projector. The distortion of the pattern when it strikes objects is captured by an infrared camera that is offset from the projector.

The camera uses triangulation principles to compute distance by examining the pattern's displacement (parallax) from its initial shape.

#### 2. Time-of-Flight (ToF) Mechanism:

The whole field of view is covered by a quick pulse of light (which is a phase-modulated signal), often infrared, that is emitted by the camera. The optical sensor calculates the precise amount of time it takes for light to move from the source to the item and return to the receiver after reflecting off of it. The intensity and phase of the returned signal is captured at every pixel. The distance is computed by comparing the current phase modulation with the received (reflected) signal.

### Question 2 (1 point) ✓ Saved

Compared to Lidar, which property makes Radar attractive for autonomous vehicles operating in heavy snow or rain?

- ☐ Radar waves are faster than light waves.
- ☐ Radar suffers less from traffic interferences.
- ☐ Radar offers higher accuracies in resolving distances.
- ☒ Radio waves have a longer wavelength.

### Q3

1.

An Inertial Measurement Unit (IMU) typically combines two types of sensors which include an accelerometer and a gyroscope to measure 3D motion along three orthogonal axes (x, y, z).

Therefore, three (3) accelerometers and three (3) gyroscopes are used by an IMU for the analysis of motion in each dimension.

Accelerometer: Measures linear acceleration (for which gravity is typically subtracted).

Gyroscope: Measures the rate of rotation (angular velocity).

2.

Dead reckoning is the technique of determining a robot's position using only an IMU by integrating acceleration twice (first to velocity, then to displacement). Over time, integration drift causes inaccuracies, which include the following:

Sensor Noise and Bias: All IMUs have continual biases, noise, and tiny measurement mistakes. Gyroscope drift is the accumulation of orientation errors caused by errors in angular velocity detection. Accelerometer drift is when acceleration is measured incorrectly, and thus, velocity and position computations deviate. Therefore, the IMU may detect a very slight non-zero acceleration even while the robot is motionless.

Error Compounding (Propagation of uncertainties): Even the slightest error is built up at each time step since the position is determined by integrating these signals over time.

Growth Quadratic: A modest constant bias in acceleration (for example: gravity) causes an inaccuracy that increases quadratically with time ( $t^2$ ) since the position must be found by integrating acceleration twice. The predicted location will drift considerably from the robot's real location in a brief amount of time.

**Question 4** (1 point) ✓ *Saved*

What is a potential disadvantage of using active sensors (like ultrasonic sensors or lidars) compared to passive ones?

- ☐ They cannot measure distance.
- ☒ They can suffer from interference with other sensors of the same type.
- ☐ They only measure ambient environmental energy.
- ☐ They offer lower performance than passive sensors.

**Q5**

The following two (2) factors determine the value of intensity from the LiDAR's returned pulse, which also measures the distance based on the time-of-flight:

1.

Surface Material (Reflectivity): The material characteristics of the object the pulse strikes have a significant impact on the intensity. While dark or matte surfaces absorb more energy and produce a lesser intensity, light-colored or highly reflecting surfaces return a stronger signal.

2.

Incidence Angle: The amount of light reflected back toward the sensor depends on the angle at which the laser beam strikes the surface. When a beam strikes a surface head-on or perpendicularly, it usually returns more intensity than when it strikes at a shallow, oblique angle.

## **Q6**

The sensor in a simple single-channel encoder only picks up ticks when the wheel turns, giving speed but leaving large information gaps. A quadrature encoder, on the other hand, keeps track of the two offset signals' sequence. The list of advantages for using quadrature encoding as compared to a basic single-channel optical encoder are compared as follows:

### **1. Directional Detection**

The most significant advancement over a single-channel system is directional sensing. The robot can ascertain the direction of motion by tracking which channel leads (for example, if A rises before B, the wheel is rotating clockwise; if B rises before A, it is turning counterclockwise). The same ticks would be reported by a single-channel encoder whether the robot is travelling forward or backward.

### **2. Increased Resolution**

Each physical slit in the encoder disk has four times as many state changes (edges) to count due to the two signals (rising A, rising B, falling A, falling B). The robot can measure much smaller movement increments due to this 4x encoding, which improves positioning accuracy and facilitates more seamless velocity control.

### **3. Vibration and Noise Reduction**

False counts brought on by mechanical vibration can be eliminated with the use of quadrature signals. A quadrature system will see the direction switch back and forth, accurately netting a zero change in position, while a single-channel encoder may register a motor's tiny vibrations as continuous forward motion.

## **Quiz 2**

**Question 1** (1 point) ✓ *Saved*

The inverse differential kinematics of a mobile robot is used to compute:

- ☐ The robot's body-frame velocity given its wheel angular speeds
- ☐ The robot's world-frame velocity given its wheel angular speeds
- ☒ The wheel angular speeds required given its desired body-frame velocity
- ☐ The wheel angular speeds required given its desired world-frame velocity

The planar velocity of a typical stationary (non-steering) wheel is subject to two separate kinematic limitations. A conventional wheel has only two degrees of freedom (dof), which restricts its potential motion and velocities based on its physical orientations.

1. Rolling without sliding (forward rolling) Constraint - Longitudinal:

The wheel must roll in its main direction of motion without sliding. It establishes a connection between the wheel's linear velocity and rotational speed.

Thus, meaning that the wheel's radius multiplied by its angular velocity ( $v = r * \phi$ ) must precisely equal the wheel center's velocity along its forward axis. This guarantees that the immediate velocity of the contact point with respect to the ground is zero (no longitudinal slip).

2. No sideways Sliding Constraint - Lateral:

This restriction acts as a no-skid regulation, preventing the wheel from moving sideways. It stipulates that the wheel's velocity component perpendicular to its rotational plane must be zero.

Thus, meaning that the fixed standard wheel cannot move laterally along its axle (centre of rotation), in contrast to spherical or Swedish wheels, which have three degrees of freedom and can go in any direction. The robot's range of mobility at any given time is limited by this non-holonomic limitation. This implies that the perpendicular velocity must be zero, therefore no lateral skidding.

rolling without sliding (first wheel)

rolling without sliding (second wheel)

no sideways sliding (first wheel)

no sideways sliding (second wheel)

## Forward Differential Kinematics

$$\begin{bmatrix} v_x \\ \omega \end{bmatrix} = \frac{1}{2} \begin{bmatrix} r & r \\ r/b & -r/b \end{bmatrix} \begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \end{bmatrix}$$

For Question 3 of Quiz 2, please use the **closed-form solution** derived in class yesterday when computing the robot's updated pose.

we derived the **closed-form solution** for the differential-drive motion model, showing that the robot follows constant-curvature arcs (see Slide 32 of L04). Today, we implemented a **numerical approximation** of this model for dead reckoning.

$$1. \begin{aligned} v_x &= (1/2)[0.1(2+3)] = (1/2)(0.5) = 0.25\text{m/s} \\ \omega &= (1/2)[(0.4)(2-3)] = (1/2)(-0.4) = -0.2 \text{ rad/s} \end{aligned}$$

$$2. \begin{aligned} x &= \{0.25\}/\{-0.2\}(\sin(-1) - \sin(0)) = -1.25(-0.8415 - 0) = 1.05 \text{ m} \\ y &= \{0.25\}/\{-0.2\}(1 - \cos(-1)) = -1.25(1 - 0.5403) = -0.57 \text{ m} \\ \theta(t) &= \omega t - \omega(0) = (-0.2)(5) - 0 = -1.0 \text{ rad or } -57.3 \text{ degrees} \end{aligned}$$

Final Predicted Pose:  $\{x, y, \theta\} = (1.05 \text{ m}, -0.57 \text{ m}, -1.0 \text{ rad})$

3.

You wouldn't expect the robot to land precisely in the intended stance in an actual deployment. Systematic errors are small discrepancies between the theoretical model parameters and the actual wheel diameters  $r$  or the precise distance between wheels ( $2b$ ). Physical elements like wheel slip on the floor, uneven terrain, or internal motor friction are examples of non-systematic errors. During the integration process, tiny inaccuracies in velocity measurement add up over a period of five seconds, resulting in a greater disparity in the final location (this may be due to the integration of drift).

1. Distance/Time =  $(2\pi R) / T = (2\pi \cdot 2) / 10 = (4\pi) / 10 = 0.4\pi = 1.257 \text{ m/s}$

$w = (-2\pi \text{ rad}) / 10 \text{ s} = -0.2\pi = -0.628 \text{ rad/s}$

NOTE: The angular velocity is negative because the robot's path is rotating clockwise.

2.

Right Wheel ( $\phi_1$ ):

$\phi_1 = (1/0.1)(0.4\pi + (-0.2\pi)(0.25)) = 10(0.4\pi - 0.05\pi) = 10(0.35\pi) = 3.5\pi$   
 $= 10.996 \text{ rad/s}$

Left Wheel ( $\phi_2$ ):

$\phi_2 = (1/0.1)(0.4\pi - (-0.2\pi)(0.25)) = 10(0.4\pi + 0.05\pi) = 10(0.45\pi) = 4.5\pi$   
 $= 14.137 \text{ rad/s}$

3.

The turning geometry necessary for a circular path causes the wheel's angular velocities to vary as the outer wheel needs to spin more quickly. Since the two wheels follow routes of varying lengths to preserve the circular arc, the wheel angular velocities differ:

Right Wheel ( $\phi_1$ ) - Inside:

The right wheel is situated on the curve's inner circumference. It spans a shorter distance (a circle with radius  $R-b$ ) and hence must spin slower to avoid slipping or forcing the robot off the route.

Left Wheel ( $\phi_2$ ) - Outside:

The left wheel is on the curve's outer circumference. Since the robot is rotating towards the right (clockwise), it must spin faster in order to travel a greater distance (a circle with radius  $R+b$ ) in the same amount of time (10 seconds).

The necessary body linear velocity  $v_x$  (1.257 m/s) is precisely equal to the average speed of the two wheels  $(r/2)(\phi_1 + \phi_2)$ .

# Quiz 1

Proprioceptive sensors are internal robotic sensors. These sensors act as self-analytical measurements to record the robot's position, velocity of the wheels, acceleration of the body, internal temperature, and force (such as torque). Some applications of proprioceptive sensors allow for closed-loop control (feedback), motion control, stability, force control, and navigation. Some examples include encoders (along the shaft/wheel axle), force/torque sensors (to measure weight/load), and battery monitors.

Exteroceptive sensors are external robotic sensors, meaning they calculate information from the robot's environment. These sensors look outward to perceive objects, distances, and external conditions (such as mapping, sound, autonomous navigation, and obstacle avoidance).

Exteroceptive sensors measure distance to obstacles, light intensity, sound, and the presence of objects. They are essential for tasks like navigation, perception, and interactions with objects. Some examples of exteroceptive sensors include LiDAR, Ultrasonic Sensors (measure distance through light or sound), cameras (capture visual data to identify the surroundings), GPS (determines the robot's position relative to the Earth), or an Inertial Measurement Unit (IMU) for orientation and acceleration monitoring.

$$T_{IR} = \begin{bmatrix} \cos(-30) & -\sin(-30) & 5 \\ \sin(-30) & \cos(-30) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$T_{RI} = T_{IR}^{-1} = \begin{bmatrix} R_{IR} & P_{IR} \\ 0 & 1 \end{bmatrix}^{-1}$$

$$= \begin{bmatrix} \cos(-30) & \sin(-30) & -\cos(-30)5 - \sin(-30)0 \\ -\sin(-30) & \cos(-30) & \sin(-30)5 - \cos(-30)0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$T_{RI} = \begin{bmatrix} 0.866 & -0.5 & -4.33 \\ 0.5 & 0.866 & -2.5 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} P_{IL} \\ 1 \end{bmatrix} = T_{RI} \begin{bmatrix} P_{RL} \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.866 & -0.5 & -4.33 \\ 0.5 & 0.866 & -2.5 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.866(2) & -0.5(2) & -4.33(1) \\ 0.5(2) & 0.866(2) & -2.5(1) \\ 1 & 1 & 1 \end{bmatrix}$$

$$| P_{IL} | = \begin{vmatrix} 0.866(2) & -0.5(2) & -4.33(1) \\ 0.5(2) & 0.866(2) & -2.5(1) \\ 1 & & \end{vmatrix}$$

$$| P_{IL} | = \begin{vmatrix} 0.866(2)-0.5(2)-4.33(1) \\ 0.5(2)+0.866(2)-2.5(1) \\ 1 \end{vmatrix}$$

$$| P_{IL} | = \begin{vmatrix} -3.5979 \\ 0.232 \\ 1 \end{vmatrix}$$

Therefore  $x = -3.6$  m and  $y = 0.23$  m for the coordinates of the landmark in the global reference frame.

**Question 1** (1 point) ✓ *Saved*

In mobile robotics, there is generally a trade-off between maneuverability and controllability. Highly maneuverable robots are typically more difficult to control. True or false?

- ☒ True
- ☐ False

**Question 3** (1 point) ✓ *Saved*

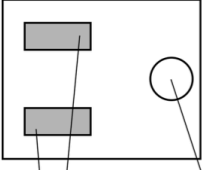
How many wheels are at minimum required to achieve static stability?

- ☐ Two wheels in any configuration
- ☒ Two wheels, if the center of mass is below the wheel axle
- ☐ Three wheels in any configuration
- ☐ Three wheels, if the center of mass is within the triangle spanned by the wheel contacts

Question 4 (2 points) ✓ Saved

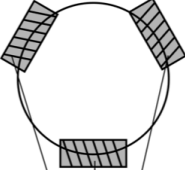
The image shows three different wheel configurations for mobile robots (A, B, C). Order the wheel configurations based on the resulting maneuverability of the mobile robot platform from 1 - lowest to 3 - highest.

**A**



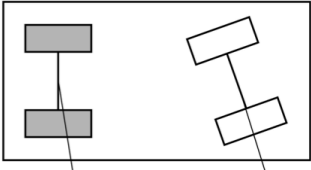
individually powered, not steered      passive spherical wheel

**B**



powered 45° swedish wheel

**C**



connected, powered, not steered      connected, unpowered, actively steered

1    ▾

C

2    ▾

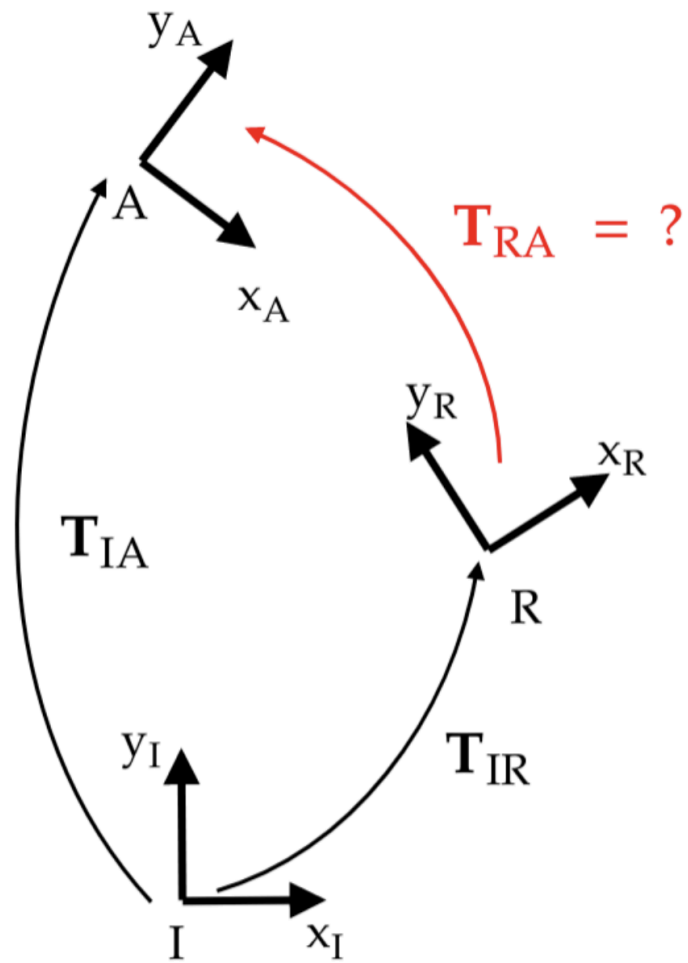
A

3    ▾

B

Assume we know the transformations  $\mathbf{T}_{IR}$  and  $\mathbf{T}_{IA}$ . How can we compute the transformation  $\mathbf{T}_{RA}$ ?

Recall:  $\mathbf{T}_{AB}$  transforms vectors from frame **B** to frame **A**.



☐  $T_{RA} = T_{IR} T_{IA}$

☐  $T_{RA} = T_{IA}^{-1} T_{IR}$

☒  $T_{RA} = T_{IR}^{-1} T_{IA}$

☐  $T_{RA} = T_{IA} T_{IR}$